Detecting Structural Breaks in Crypto Markets

A beginner-friendly empirical report inspired by structural-break methods in *Advances in Financial Machine Learning*

Prepared from Binance daily OHLC market data

Generated on 2026-07-08

What this report does. We take historical crypto price data for BTC, ETH, ETC, SOL, and HYPE and apply structural-break diagnostics: a BDE-inspired recursive forecast-error CUSUM, Chu-Stinchcombe-White CUSUM, Chow-type Dickey-Fuller, SADF, QADF, a conditional-tail ADF summary, and sub/super-martingale trend tests (SM-Exp, SM-Power, SM-Poly1, SM-Poly2), plus a walk-forward Gaussian hidden Markov model (HMM) regime filter. The goal is educational: show how the tests behave, how to interpret the plots, and why one should treat flagged regimes as research features rather than automatic trading signals.

Contents

1 Intuition

A **structural break** is a change in the data-generating mechanism. In markets this often means that a process that looked stable, noisy, or mean-reverting starts behaving like a momentum process, an explosive bubble, or a fast collapse. These breaks can be useful because many market participants adapt slowly, but the diagnostics are not crash predictors. A strong reading says: *something about the recent price path is unusual under the reference model.* That unusual behavior may be a bubble, a burst, a volatility shock, a liquidity episode, a listing effect, a leverage unwind, or a false alarm.

2 Data and Parameters

The raw source data are Binance daily OHLCV klines. The report uses the largest available Binance spot USDT history where spot is available. HYPEUSDT is not available on Binance spot in the API check used here, so HYPE falls back to Binance USD-M futures daily klines. All diagnostics are computed on daily closes through 2026-07-07.

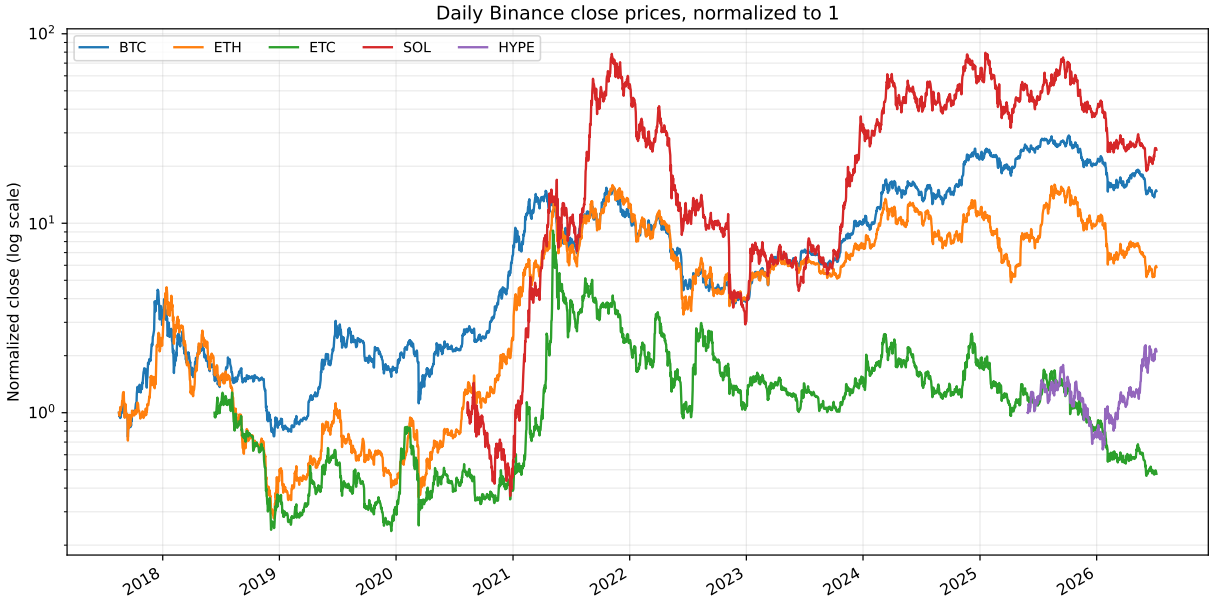

Figure 1: Downloaded daily close prices, normalized to 1 at each asset's first available row. A log scale is used because crypto prices can move by orders of magnitude.

Asset	Source	Raw start	Raw end	Raw rows	τ	SDFC candidate	SDFC t	Max SADF	Max CSW	Max SMT	Runs
BTC	Binance spot	2017-08-17	2026-07-07	3247	488	2019-02-08	1.38	2021-01-08	2020-03-12	2025-10-28	16
ETH	Binance spot	2017-08-17	2026-07-07	3247	488	2020-03-13	0.90	2021-05-11	2020-03-12	2021-12-09	18
ETC	Binance spot	2018-06-12	2026-07-07	2948	443	none (non-positive t)	-0.26	2021-05-06	2020-03-12	2021-11-17	6
SOL	Binance spot	2020-08-11	2026-07-07	2157	324	2022-12-30	0.59	2023-12-25	2022-11-09	2022-01-04	10
HYPE	Binance USD-M futures	2025-05-30	2026-07-07	404	61	2026-01-21	1.71	2026-06-01	2026-01-28	2025-12-28	7

Table 1: Daily Binance data and headline diagnostic results. τ is the minimum daily window length used in recursive tests. Runs count retrospective consensus alert runs, where at least two diagnostic families were simultaneously in their research-alert region.

Parameter	Value
Transaction cost	0.10% per exposure change
Online signal quantile	0.95 prior expanding quantile
Minimum online history	60 finite observations
Momentum gate	trailing 20-day log return must be positive
Holding window	20 calendar daily observations after an alert
Minimum diagnostic window	$\max(60, \lceil 0.15n \rceil)$
HMM regime model	3 states, refit every 126 obs, min 365 training obs
ADF Monte Carlo calibration	2000 random-walk simulations, seed 123
Walk-forward validation	expanding 730-day train, 180-day test blocks

Table 2: Backtest and diagnostic parameters used by the generated report.

3 Diagnostic Families

The table below maps each statistic in this report to the source material it is based on, so readers can connect the empirical output back to the original definitions.

Report statistic	Source	Fidelity
BDE-inspired CUSUM	AFML §17.3.1 (Brown-Durbin-Evans 1975)	inspired, online variant
Two-sided CSW excess	AFML §17.3.2 (Chu-Stinchcombe-White 1996)	two-sided heuristic
SDFC	AFML §17.4.1 (Chow-type Dickey-Fuller)	faithful, zero-lag
SADF	AFML §17.4.2 (Phillips-Wu-Yu 2011)	faithful, zero-lag
QADF / CADF	AFML §17.4.2.4–17.4.2.5	faithful, zero-lag
SMT score	AFML §17.4.3 (SM-Exp/Power/Poly1/Poly2)	all four forms, one φ
HMM regime filter	Hamilton (1989); Cartea et al. Ch. 12	walk-forward, filtered

Table 3: Where each diagnostic comes from and how closely it follows the source definition.

Complete formal definitions of every statistic, the alert rule, the long/flat backtest, the finite-sample calibration null, and the validation checks are collected in Appendix A, and the reference Python that computes them is reproduced verbatim in Appendix B. Together they are sufficient to recompute every number in this report from the raw daily closes.

3.1 CUSUM Tests

CUSUM means cumulative sum. The BDE-inspired series in this report is a recursive one-step forecast-error CUSUM from a simple log-price trend model. At each close t the trend model is refitted on all data before t, and the standardized forecast error

$$\hat{\omega}_t = \frac{y_t - \hat{\alpha}_{t-1} - \hat{\beta}_{t-1} t}{\hat{\sigma}_{t-1}\sqrt{1 + h_t}},$$

where h_t is the regression leverage of the new point, is accumulated into a scaled running sum. It is **not** a formal Brown-Durbin-Evans boundary test: residuals are normalized online with shifted expanding statistics so a value at close t uses only information available through t.

The Chu-Stinchcombe-White statistic asks whether log price has moved too far from an earlier reference level relative to realized volatility:

$$S_{n,t} = \frac{y_t - y_n}{\hat{\sigma}_t\sqrt{t - n}}, \qquad \hat{\sigma}_t^2 = \frac{1}{t-1}\sum_{i=2}^{t}(\Delta y_i)^2.$$

The report applies the level-departure statistic symmetrically to bubbles and crashes. Because of that absolute-value transformation, the displayed CSW excess should be read as a two-sided heuristic rather than the original one-sided 5% test.

3.2 ADF-Style Tests

SDFC scans for a single full-sample unknown break into an explosive alternative. It is not traded here because choosing the break date from the full sample would use future data. SADF, QADF, and CADF scan repeated backward windows ending at each date. The implementation uses the zero-lag ADF form for readability and speed:

$$\Delta y_t = \alpha + \beta y_{t-1} + \varepsilon_t.$$

For formal interpretation, the report now adds cached finite-sample random-walk simulations for SADF/QADF/CADF critical values. The full-sample top 5% plot flags remain research visualizations, not p-values.

3.3 SMT-Style Trend Tests

The sub/super-martingale score evaluates the four trend specifications from the source text: an exponential trend on log price (SM-Exp), a power trend of log price on log time (SM-Power), and quadratic trends on normalized raw price (SM-Poly1) and log price (SM-Poly2). For each specification the trend coefficient $\hat{\beta}_{t_0,t}$ is estimated on backward windows and the penalized absolute t-statistic is aggregated as

$$\mathrm{SMT}_t = \sup_{t_0 \in [1, t-\tau]} \left\{ \frac{\left|\hat{\beta}_{t_0,t}\right|}{\hat{\sigma}_{\beta_{t_0,t}} (t - t_0)^{\varphi}} \right\}, \qquad \varphi = 0.5,$$

where the penalty $(t - t_0)^{\varphi}$ prevents long windows from dominating through mechanically large t-values. High values flag unusually strong trend-like growth or collapse. They are useful features, but the reported score is a compact heuristic rather than a complete production SMT battery across many φ values.

3.4 Hidden Markov Regime Model

In addition to the structural-break statistics, the report fits a Gaussian hidden Markov model (HMM) on two causal daily features: the one-day log return and a trailing multi-day log momentum. The HMM is a different kind of regime tool: instead of testing for a break against a reference model, it assumes the market switches between a small number of latent states s_t following a Markov chain with transition matrix A, and that the observed feature vector is drawn from a state-specific Gaussian,

$$x_t \mid s_t = k \sim \mathcal{N}(\mu_k, \Sigma_k), \qquad \Pr[s_t = k \mid x_{1:t}] \propto \Big(\textstyle\sum_j \Pr[s_{t-1} = j \mid x_{1:t-1}]\, A_{jk}\Big)\, \mathcal{N}(x_t; \mu_k, \Sigma_k).$$

The right-hand recursion is the *filtered* state probability: it conditions only on observations up to t. This latent-state view goes back to Markov-switching models in econometrics and connects to the Markov-chain regime models used for order-flow states in the algorithmic-trading literature.

The implementation is designed so that no forward-looking information can reach the trading signal:

- Parameters are re-estimated walk-forward on an expanding window at fixed refit dates; the model used at close t was fitted only on data before its refit date, which is at or before t.
- The regime at close t is the argmax of *filtered* state probabilities from a causal forward pass over observations up to t. Smoothed (forward-backward) probabilities are never used out of sample, because smoothing conditions on future observations.

- The mapping from state index to bullish/non-bullish is computed from the training window only, by the state-weighted mean of same-window daily returns. If no state has a positive mean return, the signal stays flat.
- Like every other strategy in this report, the position is shifted by one day before it trades.

Assets with less history than the minimum training window (currently HYPE) remain flat and report an inactive HMM. The EM fit uses deterministic quantile-based initialization, so results are exactly reproducible without random seeds.

4 Formal Inference Versus Research Features

Statistic	Threshold type	Formal p-value?	Online?
BDE-inspired CUSUM	shifted expanding residual z-score	no	yes for strategy, no for retrospective plot flags
Two-sided CSW excess	book-inspired curve applied symmetrically	heuristic	yes
SADF/QADF/CADF visual flags	full-sample top 5%	no	no
SADF/QADF/CADF critical exceedances	simulated random-walk finite-sample critical values	approximate	no, retrospective inference only
SDFC	full-sample unknown-break scan	no in this report	no
HMM regime filter	walk-forward Gaussian HMM, filtered (causal) state probabilities	no	yes
Long/flat strategies	prior expanding thresholds plus one-day execution shift	no	yes

Table 4: Interpretation labels for the report's diagnostics and strategy conversions.

5 How to Read the Plots

Each asset figure has six panels: price, BDE-inspired recursive CUSUM, two-sided CSW excess, ADF-family statistics, SMT-style trend score, and the filtered HMM regime state. Orange shading marks **retrospective consensus alert periods**: at least two diagnostic families were in their full-sample top-alert region or exceeded the fixed CSW boundary. This shading is useful for visual structure clustering, but it is not an online trading signal. In the HMM panel, green shading marks the days on which the filtered bull state was active; unlike the orange shading, this *is* the online strategy state.

The online strategy signals are separate. BDE, SADF, QADF, CADF, and SMT strategy events use prior expanding thresholds only. CSW strategy events use the current positive excess because the statistic already embeds its reference boundary. The HMM strategy holds a long position exactly while the filtered bull state is active. The filtered HMM state per day is also stored in each asset's diagnostics CSV.

6 Results by Asset

6.1 BTC

16 retrospective consensus alert run(s) were flagged on the daily clock. The first listed run begins on 2021-01-02. The SDFC candidate is 2019-02-08 with t-statistic 1.38. SADF peaks on 2021-01-08, the two-sided CSW heuristic peaks on 2020-03-12, and the SMT score peaks on 2025-10-28. The walk-forward HMM covers 2862 daily observations from 2018-09-06 and spends 749 of them in its bull state.

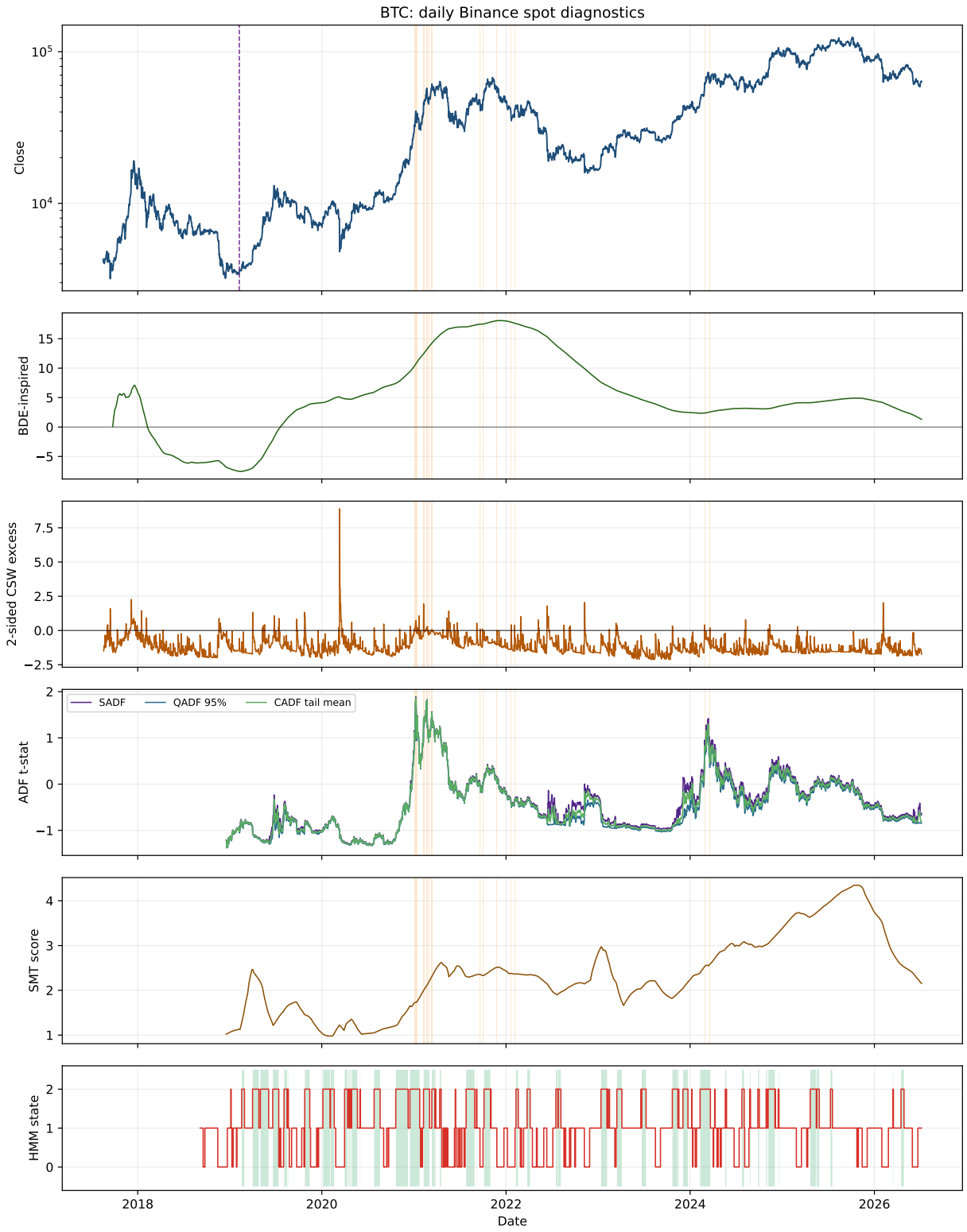

Figure 2: BTC structural-break diagnostics. Orange shading is retrospective consensus, not an online trading signal. Green shading in the bottom panel marks the filtered HMM bull state, which is the online HMM strategy state.

6.2 ETH

18 retrospective consensus alert run(s) were flagged on the daily clock. The first listed run begins on 2021-01-04. The SDFC candidate is 2020-03-13 with t-statistic 0.90. SADF peaks on 2021-05-11,

the two-sided CSW heuristic peaks on 2020-03-12, and the SMT score peaks on 2021-12-09. The walk-forward HMM covers 2862 daily observations from 2018-09-06 and spends 762 of them in its bull state.

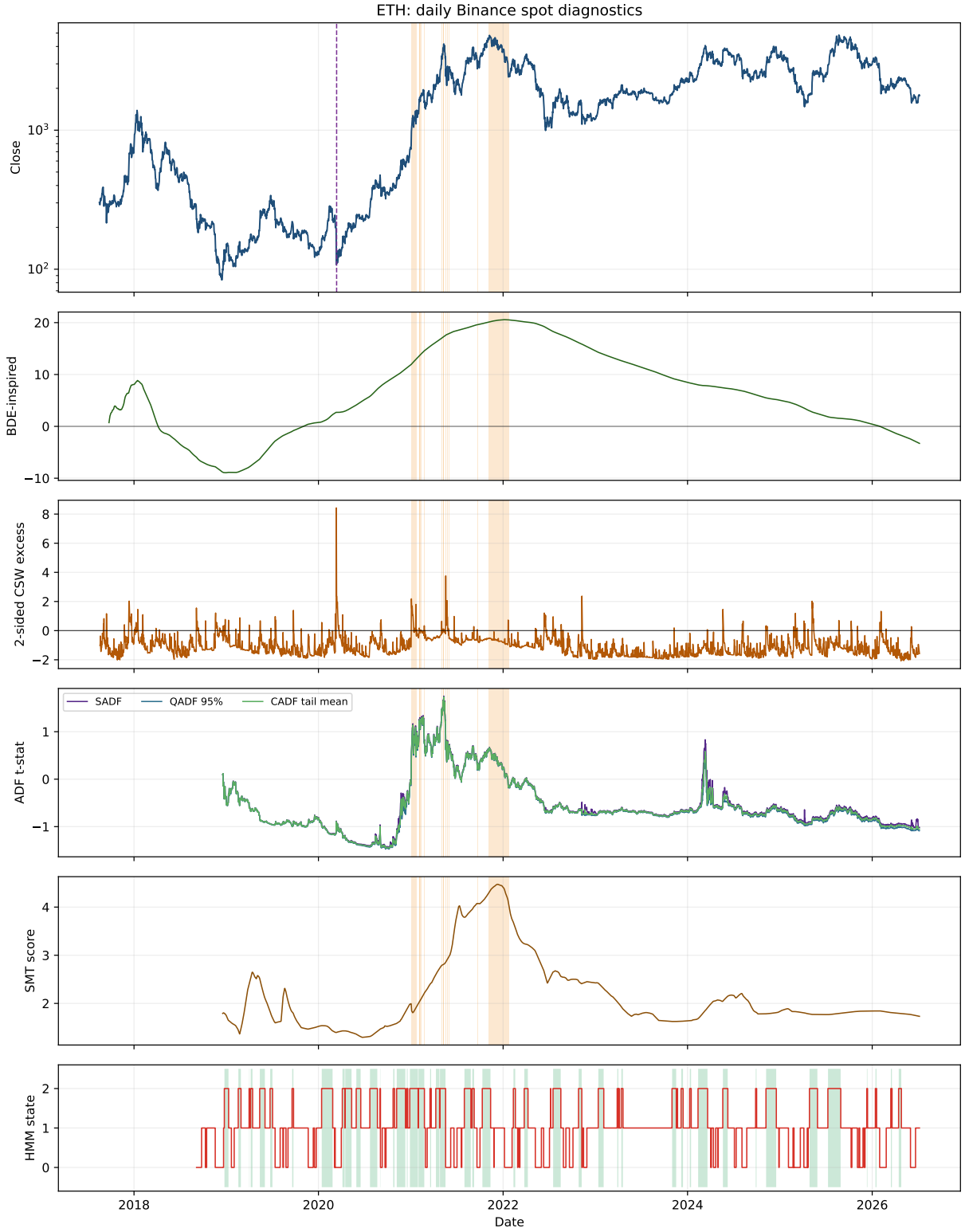

Figure 3: ETH structural-break diagnostics. Orange shading is retrospective consensus, not an online trading signal. Green shading in the bottom panel marks the filtered HMM bull state, which is the online HMM strategy state.

6.3 ETC

6 retrospective consensus alert run(s) were flagged on the daily clock. The first listed run begins on 2021-04-15. The SDFC candidate is none (non-positive t) with t-statistic -0.26. SADF peaks on 2021-05-06, the two-sided CSW heuristic peaks on 2020-03-12, and the SMT score peaks on 2021-11-17. The walk-forward HMM covers 2563 daily observations from 2019-07-02 and spends 505 of them in its bull state.

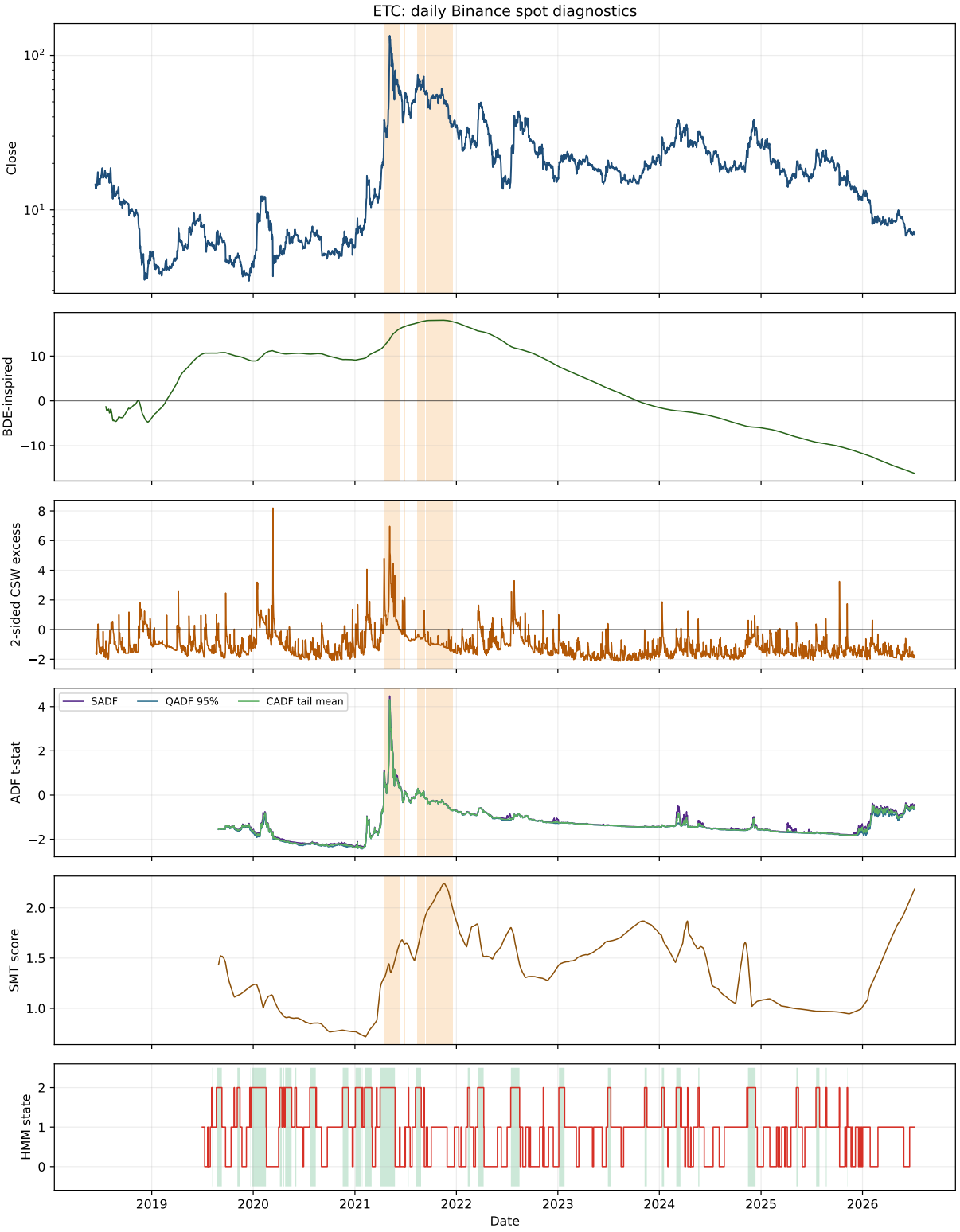

Figure 4: ETC structural-break diagnostics. Orange shading is retrospective consensus, not an online trading signal. Green shading in the bottom panel marks the filtered HMM bull state, which is the online HMM strategy state.

6.4 SOL

10 retrospective consensus alert run(s) were flagged on the daily clock. The first listed run begins on 2021-09-06. The SDFC candidate is 2022-12-30 with t-statistic 0.59. SADF peaks on 2023-12-25,

the two-sided CSW heuristic peaks on 2022-11-09, and the SMT score peaks on 2022-01-04. The walk-forward HMM covers 1772 daily observations from 2021-08-31 and spends 257 of them in its bull state.

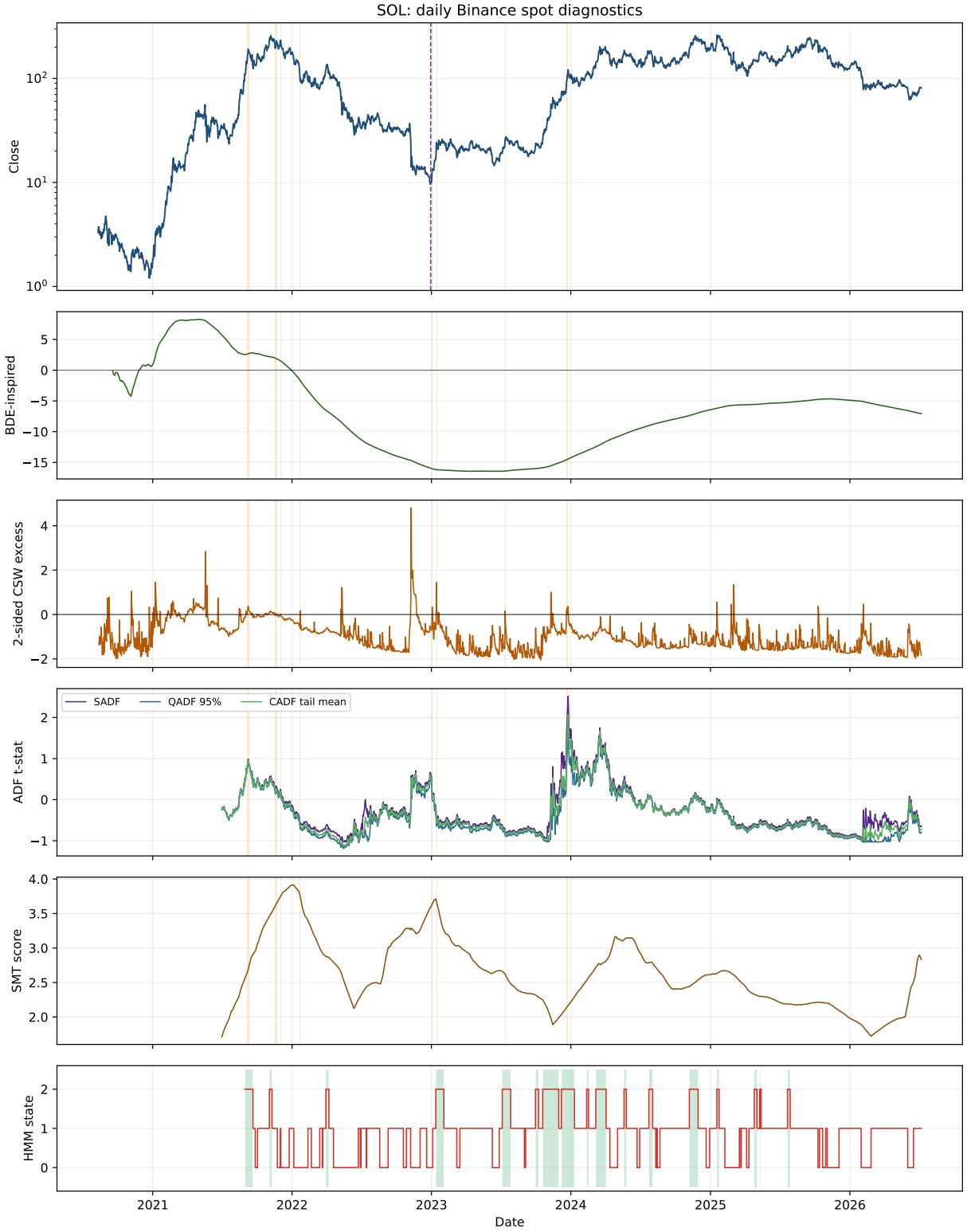

Figure 5: SOL structural-break diagnostics. Orange shading is retrospective consensus, not an online trading signal. Green shading in the bottom panel marks the filtered HMM bull state, which is the online HMM strategy state.

6.5 HYPE

7 retrospective consensus alert run(s) were flagged on the daily clock. The first listed run begins on 2025-12-18. The SDFC candidate is 2026-01-21 with t-statistic 1.71. SADF peaks on 2026-06-01, the two-sided CSW heuristic peaks on 2026-01-28, and the SMT score peaks on 2025-12-28. The walk-forward HMM covers 19 daily observations from 2026-06-19 and spends 1 of them in its bull state. HYPE uses Binance USD-M futures price klines; strategy returns exclude funding, carry, basis, and futures-specific execution costs.

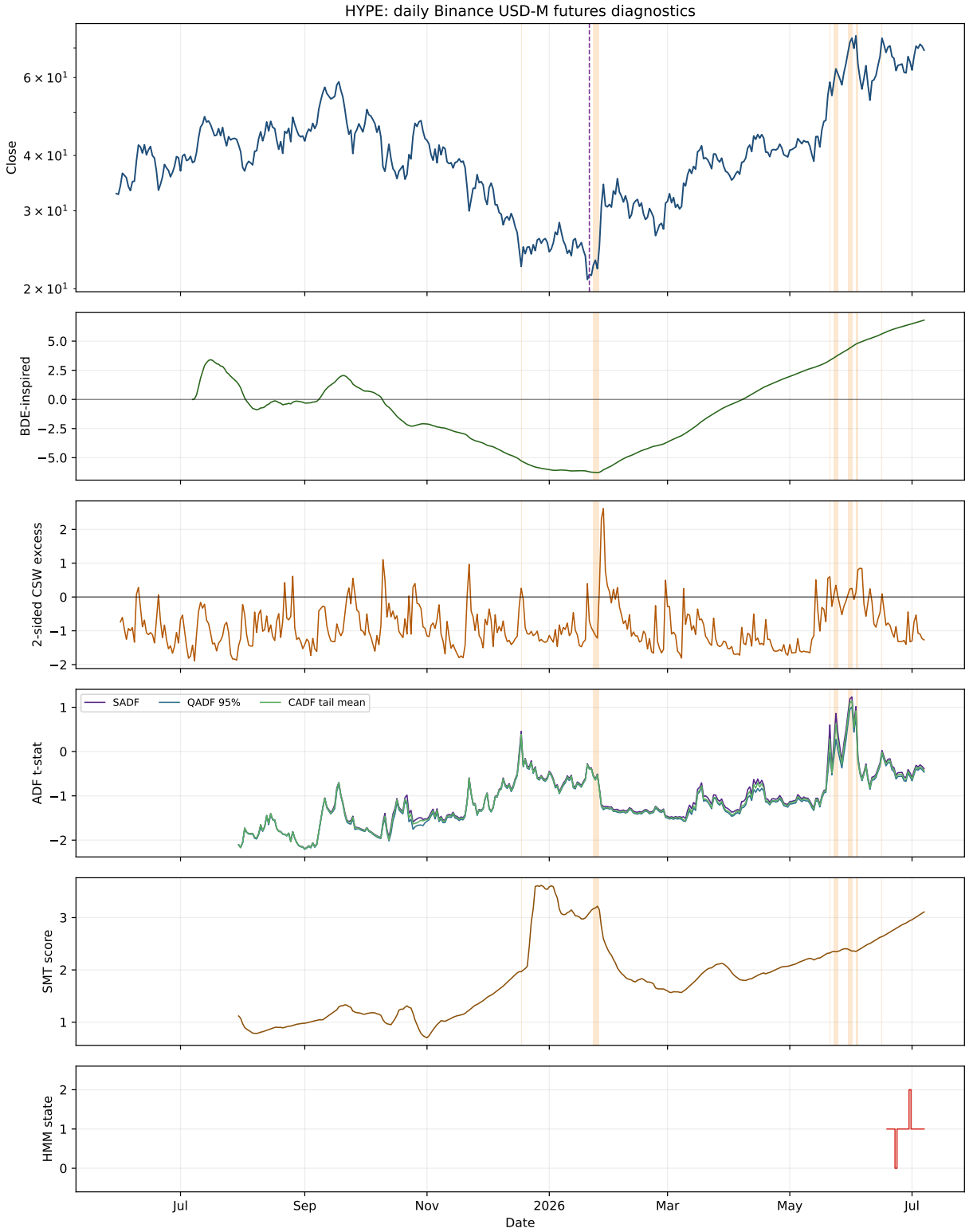

Figure 6: HYPE structural-break diagnostics. Orange shading is retrospective consensus, not an online trading signal. Green shading in the bottom panel marks the filtered HMM bull state, which is the online HMM strategy state.

7 Consensus Alert Periods

The table below lists the first few retrospective periods in which at least two diagnostic families were active. Short alert runs are common because the tests are sensitive to local trend acceleration and volatility normalization.

Table 5: First listed retrospective consensus alert periods. Period length is counted in calendar days between the first and last daily alert observation. For assets with many daily runs, the table shows the first eight runs.

Asset	Start	End	Days
BTC	2021-01-02	2021-01-03	2
	2021-01-05	2021-01-10	6
	2021-01-13	2021-01-14	2
	2021-02-08	2021-02-09	2
	2021-02-11	2021-02-11	1
	2021-02-17	2021-02-23	7
	2021-03-01	2021-03-01	1
	2021-03-11	2021-03-11	1
ETH	2021-01-04	2021-01-11	8
	2021-01-14	2021-01-14	1
	2021-01-16	2021-01-16	1
	2021-01-19	2021-01-20	2
	2021-01-24	2021-01-24	1
	2021-02-03	2021-02-03	1
	2021-02-05	2021-02-05	1
	2021-02-08	2021-02-09	2
ETC	2021-04-15	2021-06-11	58
	2021-06-13	2021-06-14	2
	2021-06-29	2021-06-30	2
	2021-08-12	2021-09-09	29
	2021-09-15	2021-09-15	1
	2021-09-20	2021-12-20	92
SOL	2021-09-06	2021-09-11	6
	2021-11-17	2021-11-17	1
	2021-11-20	2021-11-21	2
	2021-11-23	2021-11-23	1
	2021-12-01	2021-12-02	2
	2022-01-22	2022-01-22	1
	2023-01-03	2023-01-03	1
	2023-01-14	2023-01-16	3
HYPE	2025-12-18	2025-12-18	1
	2026-01-23	2026-01-26	4
	2026-05-21	2026-05-21	1
	2026-05-23	2026-05-25	3
	2026-05-30	2026-06-01	3
	2026-06-03	2026-06-04	2
	2026-06-16	2026-06-16	1

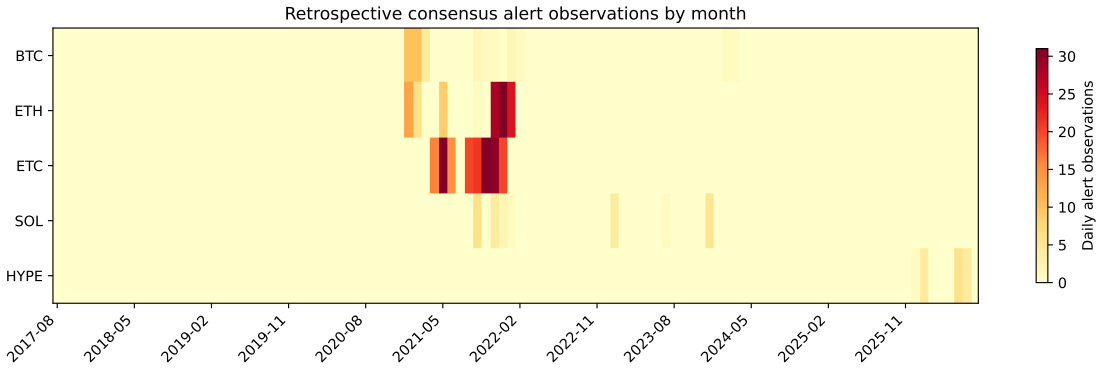

Figure 7: Retrospective consensus alert observations by month.

8 Finite-Sample Calibration and Sensitivity

The table below compares each asset's observed ADF-family maxima with cached random-walk finite-sample critical values. These simulations give a more defensible reference than the internal top 5% visual flags, although they still inherit the simplified zero-lag ADF specification.

Asset	Sims	τ	SADF 95%	Max SADF	SADF hits	QADF 95%	Max QADF	QADF hits	CADF 95%	Max CADF	CADF hits
BTC	2000	488	2.02	1.89	0	1.92	1.82	0	1.96	1.86	0
ETH	2000	488	2.02	1.75	0	1.92	1.72	0	1.96	1.73	0
ETC	2000	443	2.01	4.48	13	1.90	4.28	14	1.95	4.32	13
SOL	2000	324	2.05	2.52	4	1.94	1.61	0	1.99	2.07	1
HYPE	2000	61	2.03	1.23	0	1.91	1.01	0	1.97	1.14	0

Table 6: Simulated finite-sample critical values under random-walk null paths with matching sample length and τ. Hits count daily observations above the 95% simulated max-statistic threshold.

Table 7: Sensitivity of selected results to the minimum-window fraction. ADF-family windows vary; BDE, CSW, SMT, and HMM inputs are held at their main-run definitions.

Asset	τ rule	τ	Max SADF date	Runs	Best structure signal	Final
BTC	10%	325	2024-03-13	19	CSW	11.68x
BTC	15%	488	2021-01-08	16	CSW	11.68x
BTC	20%	650	2021-02-21	15	CSW	11.68x
ETH	10%	325	2024-03-11	21	BDE	9.35x
ETH	15%	488	2021-05-11	18	BDE	9.35x
ETH	20%	650	2021-05-11	18	BDE	9.35x
ETC	10%	295	2021-05-06	14	HMM	20.48x
ETC	15%	443	2021-05-06	6	HMM	20.48x
ETC	20%	590	2021-05-06	6	HMM	20.48x
SOL	10%	216	2023-12-25	16	CSW	27.43x
SOL	15%	324	2023-12-25	10	CSW	27.43x
SOL	20%	432	2024-03-17	15	CSW	27.43x
HYPE	10%	60	2026-06-01	7	CSW	1.62x
HYPE	15%	61	2026-06-01	7	CSW	1.62x
HYPE	20%	81	2026-06-01	7	CSW	1.62x

9 Long/Flat Backtests

The diagnostics above are research features, not trading rules. Still, it is useful to ask what a simple long-or-cash conversion would have done.

9.1 Backtest Rule

1. At each daily close t, compute diagnostics using data through t only.
2. Convert each eligible indicator into an online alert using prior expanding thresholds, except CSW, whose positive excess is already boundary-based.
3. Require the trailing momentum gate to be positive.
4. An alert computed at close t sets exposure for the close t to close $t+1$ return, assuming execution at or immediately after close t.
5. Keep the position long for the holding window after a valid alert, then move back out unless a new alert refreshes the holding window. The HMM strategy is the exception: it uses no momentum gate and no holding window, and instead stays long exactly while the filtered bull state is active.
6. Charge transaction costs whenever exposure changes. Buy-and-hold is charged one entry cost for consistency.

This is still an in-sample comparison. The best final PnL is an indicative ranking of which simple signal conversion looked promising on this sample, not evidence of a deployable strategy.

Asset	Best structure signal	Best signal final	Buy-and-hold final	Best including buy-and-hold	Max DD	Exposure	Trades
BTC	CSW	11.68x	14.77x	Buy-and-hold	-37%	17%	20
ETH	BDE	9.35x	5.86x	BDE	-61%	19%	7
ETC	HMM	20.48x	0.48x	HMM	-61%	17%	42
SOL	CSW	27.43x	24.40x	CSW	-60%	20%	9
HYPE	CSW	1.62x	2.11x	Buy-and-hold	-46%	46%	7

Table 8: Best long/flat structure-break strategy by final equity multiple. Best structure signal excludes buy-and-hold; best including buy-and-hold includes it. Max DD, exposure, and trades refer to the best structure signal.

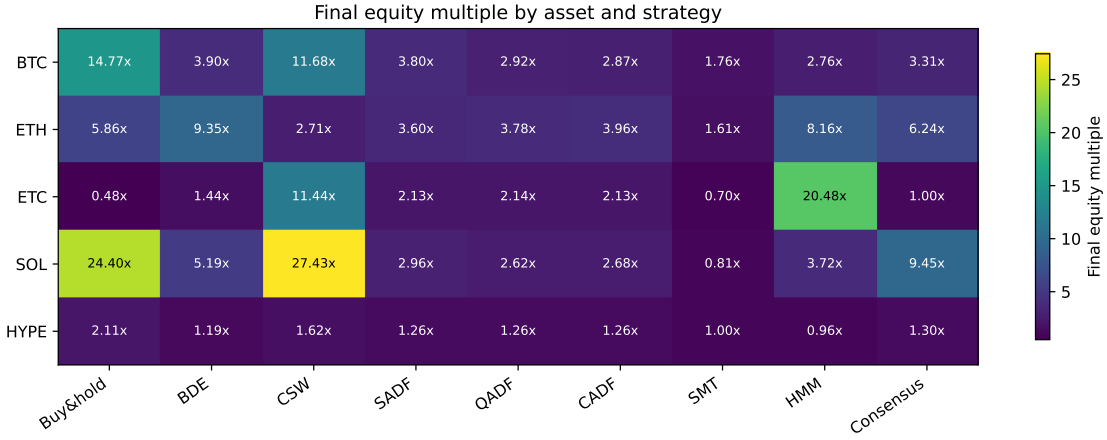

Figure 8: Final equity multiple by asset and strategy. Values are in-sample and normalized to 1 at each asset's first available observation.

9.2 PnL Curves

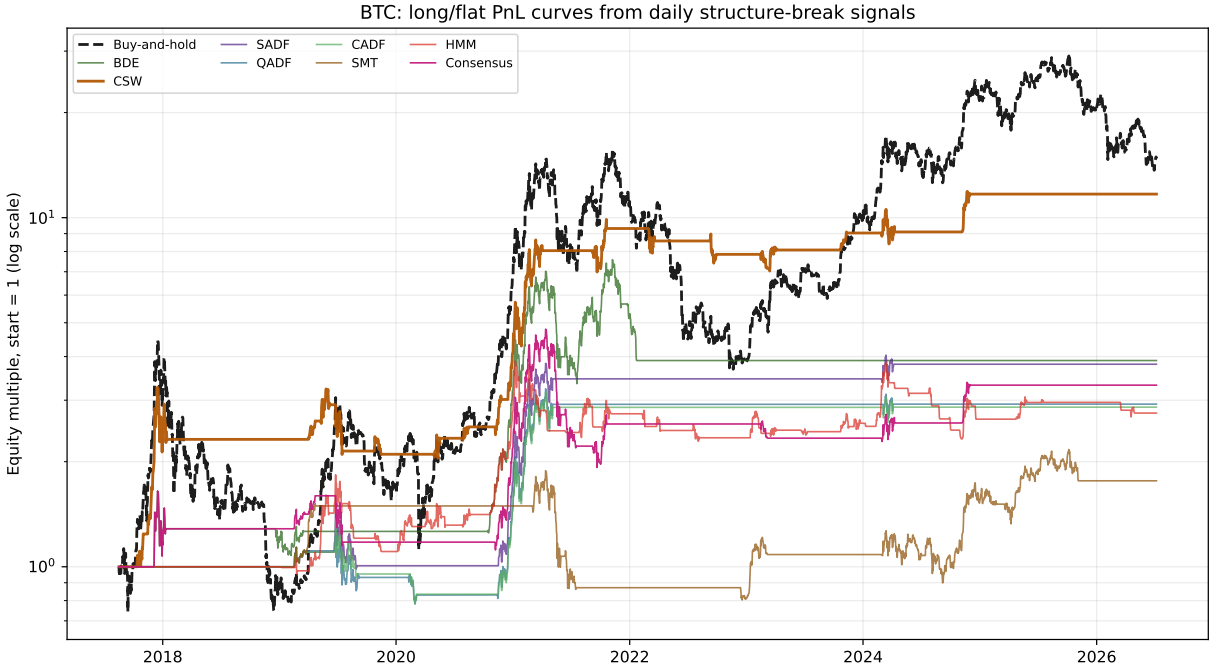

Figure 9: BTC long/flat strategy PnL curves.

Figure 10: ETH long/flat strategy PnL curves.

Figure 11: ETC long/flat strategy PnL curves.

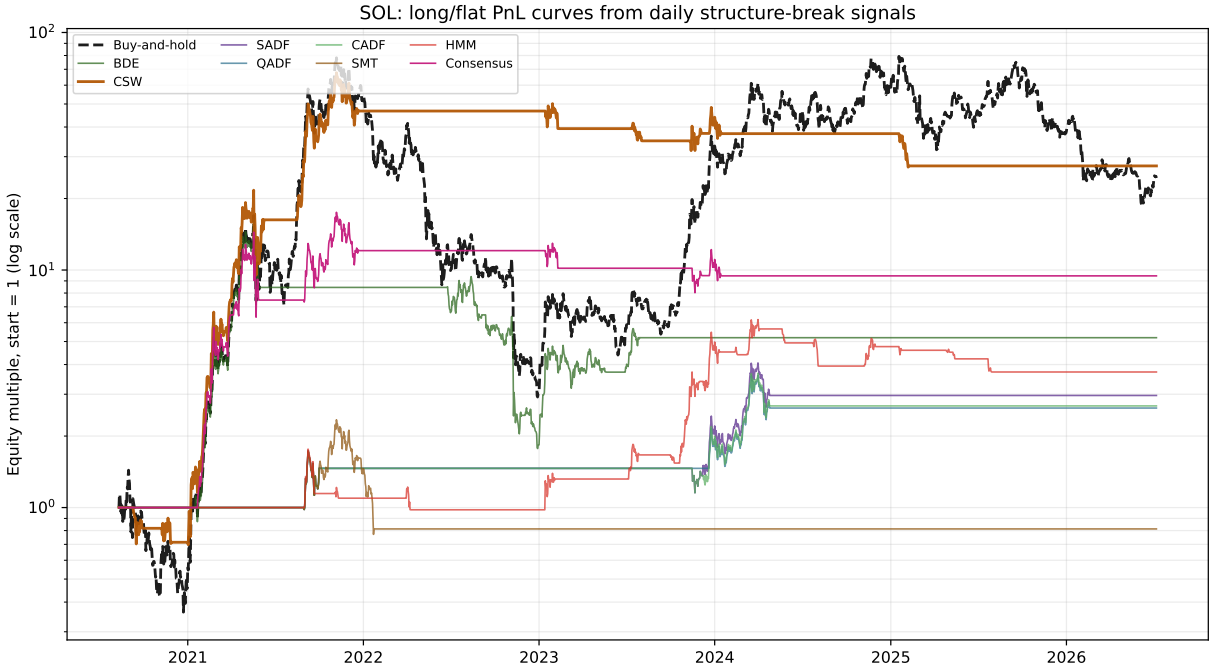

Figure 12: SOL long/flat strategy PnL curves.

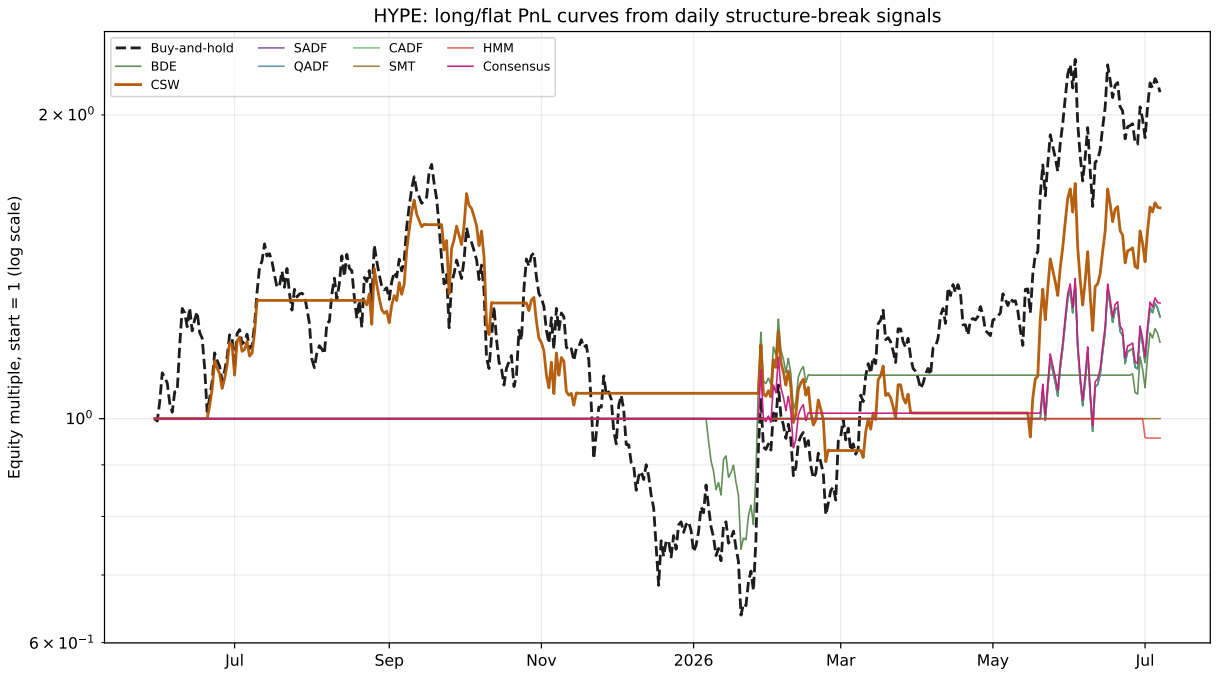

Figure 13: HYPE long/flat strategy PnL curves. HYPE uses Binance USD-M futures price data; returns exclude funding and futures-specific carry.

9.3 Full Strategy Table

Table 9: Simple long/flat strategy performance. Final multiple is the ending equity multiple from 1. CAGR is annualized over each asset's available sample. Sharpe uses daily returns and 365-day annualization. Because Sharpe is computed from arithmetic daily means, a volatile asset can show a positive Sharpe while its compounded return is negative (volatility drag); ETC buy-and-hold is an example.

Asset	Strategy	Final	Total return	CAGR	Max DD	Sharpe	Exposure	Trades
BTC	Buy-and-hold	14.77x	1377%	35%	-83%	0.79	100%	1
BTC	BDE	3.90x	290%	17%	-52%	0.62	17%	6
BTC	CSW	11.68x	1068%	32%	-37%	0.99	17%	20
BTC	SADF	3.80x	280%	16%	-31%	0.72	10%	5
BTC	QADF	2.92x	192%	13%	-36%	0.60	11%	6
BTC	CADF	2.87x	187%	13%	-41%	0.59	10%	6
BTC	SMT	1.76x	76%	7%	-57%	0.37	26%	11
BTC	HMM	2.76x	176%	12%	-41%	0.53	23%	56
BTC	Consensus	3.31x	231%	14%	-60%	0.57	15%	12
ETH	Buy-and-hold	5.86x	486%	22%	-94%	0.67	100%	1
ETH	BDE	9.35x	835%	29%	-61%	0.76	19%	7
ETH	CSW	2.71x	171%	12%	-64%	0.47	18%	22
ETH	SADF	3.60x	260%	16%	-52%	0.62	6%	2
ETH	QADF	3.78x	278%	16%	-50%	0.65	5%	1
ETH	CADF	3.96x	296%	17%	-50%	0.67	5%	1
ETH	SMT	1.61x	61%	6%	-56%	0.33	11%	5
ETH	HMM	8.16x	716%	27%	-50%	0.75	23%	47
ETH	Consensus	6.24x	524%	23%	-56%	0.68	16%	7
ETC	Buy-and-hold	0.48x	-52%	-9%	-95%	0.40	100%	1
ETC	BDE	1.44x	44%	5%	-74%	0.34	15%	7

Asset	Strategy	Final	Total return	CAGR	Max DD	Sharpe	Exposure	Trades
ETC	CSW	11.44x	1044%	35%	-67%	0.77	25%	24
ETC	SADF	2.13x	113%	10%	-61%	0.42	5%	3
ETC	QADF	2.14x	114%	10%	-61%	0.42	5%	3
ETC	CADF	2.13x	113%	10%	-61%	0.42	5%	3
ETC	SMT	0.70x	-30%	-4%	-84%	0.10	8%	6
ETC	HMM	20.48x	1948%	45%	-61%	0.89	17%	42
ETC	Consensus	1.00x	0%	0%	-78%	0.25	13%	10
SOL	Buy-and-hold	24.40x	2340%	72%	-96%	1.04	100%	1
SOL	BDE	5.19x	419%	32%	-89%	0.75	22%	6
SOL	CSW	27.43x	2643%	75%	-60%	1.14	20%	9
SOL	SADF	2.96x	196%	20%	-35%	0.72	9%	3
SOL	QADF	2.62x	162%	18%	-35%	0.69	7%	2
SOL	CADF	2.68x	168%	18%	-35%	0.67	9%	3
SOL	SMT	0.81x	-19%	-3%	-67%	0.03	7%	2
SOL	HMM	3.72x	272%	25%	-44%	0.74	12%	18
SOL	Consensus	9.45x	845%	46%	-56%	0.94	14%	6
HYPE	Buy-and-hold	2.11x	111%	96%	-64%	1.17	100%	1
HYPE	BDE	1.19x	19%	17%	-26%	0.58	13%	2
HYPE	CSW	1.62x	62%	55%	-46%	0.99	46%	7
HYPE	SADF	1.26x	26%	23%	-29%	0.75	12%	1
HYPE	QADF	1.26x	26%	23%	-29%	0.75	12%	1
HYPE	CADF	1.26x	26%	23%	-29%	0.75	12%	1
HYPE	SMT	1.00x	0%	0%	0%	n/a	0%	0
HYPE	HMM	0.96x	-4%	-4%	-4%	-0.97	0%	1
HYPE	Consensus	1.30x	30%	27%	-29%	0.76	17%	2

9.4 Walk-Forward and Multiple-Testing Checks

The walk-forward check selects the best structure signal on an expanding training window and evaluates only the next out-of-sample block. This does not prove profitability, but it reduces the most obvious winner-picking bias in the in-sample table. In the selected-signal path, repeated selections are compressed, so "CSW x6" means CSW was chosen for six consecutive blocks.

Asset	Blocks	OOS days	Selected signal path	OOS final	OOS Sharpe	OOS Max DD
BTC	13	2340	CSW x13	5.45x	1.08	-29%
ETH	13	2340	BDE x13	5.36x	0.79	-61%
ETC	12	2160	CSW x6, HMM, CSW x2, HMM x3	6.45x	0.78	-61%
SOL	7	1260	CSW x7	0.59x	-0.35	-46%
HYPE	0	0	n/a	n/a	n/a	n/a

Table 10: Expanding-window walk-forward validation of signal selection. HYPE has too little history for the default train/test split and may show no blocks.

Asset	Bootstrap sims	Best signal	Best final	Final 5–95% CI	Sharpe 5–95% CI	Reality-check p
BTC	500	CSW	11.68x	[2.53x, 83.04x]	[0.49, 1.56]	0.946
ETH	500	BDE	9.35x	[0.96x, 81.09x]	[0.21, 1.23]	0.908
ETC	500	HMM	20.48x	[1.11x, 480.46x]	[0.27, 1.44]	0.512
SOL	500	CSW	27.43x	[1.77x, 637.25x]	[0.45, 1.81]	0.958
HYPE	500	CSW	1.62x	[0.61x, 4.16x]	[-0.38, 2.31]	0.910

Table 11: Block-bootstrap uncertainty for the best in-sample structure signal and a simple centered bootstrap reality-check p-value for best excess mean return versus buy-and-hold.

The reality-check p-value estimates how often a resampled, mean-centered version of the signal family would produce a best mean daily excess return over buy-and-hold at least as large as the one observed, after accounting for the fact that the best of several signals was selected. Values near 1 mean the observed outperformance is entirely consistent with luck; only small values (for example below 0.05) would count as evidence of skill. Readers should check the table rather than assume the headline equity multiples imply significance.

10 Interpretation

Methodologically, SADF is a strong default diagnostic for repeated explosive episodes because it does not require the analyst to know the number or timing of regime switches. Empirically, in this particular in-sample long/flat conversion, the winning signal can differ by asset and must be treated as selected after looking at many alternatives.

The most important traps are multiple testing, listing-history differences, simplified zero-lag ADF regressions, and price-only data. HYPE is especially limited because it uses futures price klines without funding-rate adjustment. The HMM adds a complementary latent-state view, but its in-sample equity multiples share the same selection caveats, and its output is sensitive to the number of states and the refit schedule. Regime-change diagnostics are best read as feature-engineering tools, not as a recipe for direct trading.

11 Minimal Reproducibility Sketch

```
# online execution convention used in the backtest
event_t = signal_at_close_t
position = event_t.rolling(hold_days).max().shift(1)
strategy_return_t = position_t * close.pct_change_t - costs_t

# HMM variant: no hold window, position tracks the filtered bull state
position_hmm = (filtered_state_at_close_t == bull_state).shift(1)
```

12 Reproduce

```
python3 scripts/generate_daily_regime_chapter.py --use-cache
pdflatex -interaction=nonstopmode regime_detection_crypto_report.tex
pdflatex -interaction=nonstopmode regime_detection_crypto_report.tex
```

The generator writes daily diagnostics, strategy PnL CSVs, cached calibration files, strict JSON metadata, all figures, and this TeX report. The rendered PDF is then produced by `pdflatex`.

13 References

- Marcos López de Prado, *Advances in Financial Machine Learning*, Wiley, 2018, structural-break methods (Chapter 17).
- Álvaro Cartea, Sebastian Jaimungal, and José Penalva, *Algorithmic and High-Frequency Trading*, Cambridge University Press, 2015, Markov-chain regime models for order flow (Chapter 12), motivating the latent-regime view used by the HMM filter.
- R. L. Brown, J. Durbin, and J. M. Evans, "Techniques for testing the constancy of regression relationships over time," *Journal of the Royal Statistical Society B*, 37(2), 1975.
- C.-S. J. Chu, M. Stinchcombe, and H. White, "Monitoring structural change," *Econometrica*, 64(5), 1996; and U. Homm and J. Breitung, "Testing for speculative bubbles in stock markets," *Journal of Financial Econometrics*, 10(1), 2012.
- G. C. Chow, "Tests of equality between sets of coefficients in two linear regressions," *Econometrica*, 28(3), 1960.
- P. C. B. Phillips, Y. Wu, and J. Yu, "Explosive behavior in the 1990s Nasdaq: When did exuberance escalate asset values?" *International Economic Review*, 52(1), 2011.
- J. D. Hamilton, "A new approach to the economic analysis of nonstationary time series and the business cycle," *Econometrica*, 57(2), 1989, the canonical Markov-switching regime model.
- Binance API daily kline endpoints for BTCUSDT, ETHUSDT, ETCUSDT, SOLUSDT spot, and HYPEUSDT USD-M futures fallback.
- Local reproduction script: `scripts/generate_daily_regime_chapter.py`.

A Full Mathematical Specification

This appendix gives the complete formal definition of every statistic, the alert rule, the long/flat backtest, the finite-sample calibration, and the validation checks, so that all numbers in this report can be reproduced from the raw daily closes alone. Every structural-break detector is an implementation of a method from Marcos López de Prado, *Advances in Financial Machine Learning* (Wiley, 2018), Chapter 17, sections 17.3–17.4; the latent-regime filter follows the Markov-chain regime models of Álvaro Cartea, Sebastian Jaimungal, and José Penalva, *Algorithmic and High-Frequency Trading* (Cambridge University Press, 2015), Chapter 12, together with the Markov-switching model of Hamilton (1989). The reference Python that evaluates each expression below is reproduced verbatim in Appendix B.

Throughout, p_t denotes the daily close, $y_t = \log p_t$ the log price, and $r_t = p_t/p_{t-1} - 1$ the simple return; n is the number of observations. All detectors are *causal*: the value published at date t uses only information available at or before t. The minimum backward window is

$$\tau = \max(60,\ \lceil 0.15\, n \rceil)\,.$$

A.1 Shared regression core

Most detectors reduce to the slope t-statistic of a straight-line fit on a backward window. For a window with points $\{(x_i, z_i)\}_{i=1}^{m}$ define the centered moments

$$S_{xx} = \sum_i x_i^2 - \tfrac{1}{m}\Big(\sum_i x_i\Big)^2, \qquad S_{xz} = \sum_i x_i z_i - \tfrac{1}{m}\sum_i x_i \sum_i z_i, \qquad S_{zz} = \sum_i z_i^2 - \tfrac{1}{m}\Big(\sum_i z_i\Big)^2.$$

Then the ordinary-least-squares slope, its residual variance, and its t-statistic are

$$\hat{\beta} = \frac{S_{xz}}{S_{xx}}, \qquad \mathrm{SSE} = S_{zz} - \hat{\beta}\, S_{xz}, \qquad \hat{\sigma}^2 = \frac{\mathrm{SSE}}{m-2}, \qquad t = \frac{\hat{\beta}}{\sqrt{\hat{\sigma}^2 / S_{xx}}}.$$

Because S_{xx}, S_{xz}, S_{zz} are assembled from prefix sums, every backward window ending at t is evaluated in $O(1)$. The ADF-style tests use the zero-lag Dickey–Fuller regression

$$\Delta y_i = \alpha + \beta\, y_{i-1} + \varepsilon_i,$$

whose slope t-statistic $DF = \hat{\beta}/\mathrm{se}(\hat{\beta})$ is the quantity above with $x_i = y_{i-1}$ and $z_i = \Delta y_i$. Under a random walk $\beta = 0$; a significantly positive $\hat{\beta}$ is the explosive (mean-repelling) alternative.

A.2 BDE recursive CUSUM (AFML 17.3.1)

Following Brown, Durbin, and Evans (1975), fit the log-price trend $y = \hat{\alpha}_{t-1} + \hat{\beta}_{t-1}\,\mathrm{time}$ on all data strictly before t and form the standardized one-step-ahead recursive residual

$$\hat{\omega}_t = \frac{y_t - \hat{\alpha}_{t-1} - \hat{\beta}_{t-1}\, t}{\hat{\sigma}_{t-1}\sqrt{1 + h_t}}, \qquad h_t = \frac{1}{t} + \frac{(t - \bar{x})^2}{S_{xx}},$$

where h_t is the leverage of the new time index and $\hat{\sigma}_{t-1}$ the in-sample residual scale. The residuals are standardized online with a shifted expanding mean and standard deviation, $z_t = (\hat{\omega}_t - \mu_{<t})/\sigma_{<t}$, and accumulated into a scaled CUSUM

$$W_t = \frac{1}{\sqrt{N_t}} \sum_{i \le t} z_i,$$

with N_t the count of finite z_i through t. A large $|W_t|$ marks a persistent, one-sided drift of forecast errors, i.e. a change in the trend regime.

A.3 Chu–Stinchcombe–White excess (AFML 17.3.2)

At each date t, and for every earlier reference point $n < t$, measure how far log price has travelled from its level at n, standardized by realized volatility:

$$S_{n,t} = \frac{y_t - y_n}{\hat{\sigma}_t \sqrt{t - n}}, \qquad \hat{\sigma}_t^2 = \frac{1}{t} \sum_{i \le t} (\Delta y_i)^2.$$

Chu, Stinchcombe, and White (1996) monitor $S_{n,t}$ against a time-varying boundary; here the reported statistic is the supremum *excess* over that boundary, applied symmetrically to bubbles and crashes,

$$\mathrm{CSW}_t = \max_{n<t} \Big(|S_{n,t}| - c_{n,t} \Big), \qquad c_{n,t} = \sqrt{4.6 + \ln(t - n)}\,,$$

where 4.6 is the asymptotic 5% constant of the test. Values $\mathrm{CSW}_t > 0$ indicate the boundary was crossed.

A.4 ADF family: SADF, QADF, CADF (AFML 17.4.2)

Fix the endpoint t and run the zero-lag ADF regression on every backward window $[s, t]$ of length at least τ, producing the set of Dickey–Fuller slope t-statistics

$$\mathcal{D}_t = \big\{\, DF_{s,t} : 0 \le s \le t - \tau \,\big\}.$$

The three summaries collapse this set in different ways:

$$\mathrm{SADF}_t = \sup_s DF_{s,t}, \qquad \mathrm{QADF}_t = Q_{0.95}\big(\mathcal{D}_t\big), \qquad \mathrm{CADF}_t = \mathrm{mean}\,\big\{\, d \in \mathcal{D}_t : d \ge Q_{0.95}(\mathcal{D}_t) \,\big\},$$

i.e. the supremum ADF of Phillips, Wu, and Yu (2011), the 95th percentile of the window statistics, and the conditional mean of the upper 5% tail. All three use τ as the minimum window.

A.5 SMT trend battery (AFML 17.4.3)

For each backward window $[t_0, t]$ (length $m = t - t_0 + 1$) fit four trend specifications and take the penalized absolute trend t-statistic:

- SM-Exp: linear regression of y (log price) on time,
- SM-Power: linear regression of y on log(time),
- SM-Poly2: quadratic regression of y on time and time^2,
- SM-Poly1: quadratic regression of the normalized raw price p_t/p_0 on time and time^2,

where the quadratic t-statistic is that of the leading (curvature) coefficient. The score aggregates all four forms and all windows,

$$\mathrm{SMT}_t = \sup_{t_0 \in [1,\, t-\tau]} \; \max_{f \in \{\mathrm{Exp,Power,Poly1,Poly2}\}} \left\{ \frac{|\hat{\beta}^{(f)}_{t_0,t}|}{\hat{\sigma}_{\beta^{(f)}_{t_0,t}} (t - t_0)^{\varphi}} \right\}, \qquad \varphi = 0.5,$$

the length penalty $(t - t_0)^{\varphi}$ preventing long windows from dominating through mechanically large t-values.

A.6 SDFC break test (AFML 17.4.1)

The supremum Dickey–Fuller–Chow statistic scans every candidate break date t^* and tests a single switch into an explosive regime, reporting the largest break t-statistic over the sample. Because the break date is chosen using the full sample, it is a retrospective diagnostic and is *not* converted into a traded strategy in this report.

A.7 Hidden Markov regime filter (Cartea et al. Ch. 12; Hamilton 1989)

A Gaussian HMM is fitted on two causal daily features, the one-day log return and the trailing 20-day log momentum, $x_t = \big(y_t - y_{t-1},\ y_t - y_{t-20}\big)$. The market is assumed to switch between $K = 3$ latent states following a Markov chain with transition matrix A, with state-conditional Gaussian emissions

$$x_t \mid s_t = k \sim \mathcal{N}(\mu_k, \Sigma_k), \qquad \Pr[s_t = k \mid x_{1:t}] \propto \Big(\textstyle\sum_j \Pr[s_{t-1} = j \mid x_{1:t-1}]\, A_{jk} \Big) \mathcal{N}(x_t; \mu_k, \Sigma_k).$$

The right-hand recursion is the *filtered* state probability, conditioning only on observations up to t. Parameters are estimated by deterministic Baum–Welch EM: the E-step computes responsibilities $\gamma_t(k) = \Pr[s_t = k \mid x_{1:T}]$ and pair marginals by the forward–backward algorithm; the M-step re-estimates

$$\mu_k = \frac{\sum_t \gamma_t(k)\, x_t}{\sum_t \gamma_t(k)}, \qquad \Sigma_k = \mathrm{diag}\frac{\sum_t \gamma_t(k)\, (x_t - \mu_k)^2}{\sum_t \gamma_t(k)}, \qquad A_{jk} \propto \sum_t \xi_t(j,k),$$

with a variance floor and quantile-based initialization (so no random seed is needed). The fit is refit walk-forward every 126 days on an expanding window (minimum 365 observations). The "bull" state is the one with the highest EM-weighted mean return on the training window, used only if that mean is positive; the regime at t is the argmax of the filtered probabilities, and the signal is long while the state equals the bull state.

A.8 Consensus

The consensus detector fires only when at least two independent families agree on the same day:

$$\text{Consensus}_t = \mathbb{1}\left\{ \sum_{m \in \{\text{BDE, CSW, SADF, SMT}\}} \text{alert}_{m,t} \;\geq\; 2 \right\},$$

where $\text{alert}_{m,t} \in \{0,1\}$ is the (momentum-gated) alert of member m defined next. The QADF, CADF, and HMM signals are not part of the vote.

A.9 Alert rule and signal pipeline

Every structural-break statistic g_t is turned into a binary alert by a purely causal, self-referential threshold: the alert fires when the statistic exceeds the 95th percentile of its own strictly-prior history,

$$\text{alert}_t = \mathbb{1}\{\; g_t > Q_{0.95}\big(\{g_i\}_{i<t}\big) \;\} \;\wedge\; \mathbb{1}\{\, y_t - y_{t-20} > 0 \,\},$$

using an expanding window with a 60-observation warm-up, shifted by one day so no contemporaneous information leaks. The second factor is the 20-day momentum gate. (CSW uses its own zero boundary $\text{CSW}_t > 0$ in place of the quantile; the HMM uses its filtered bull-state indicator directly.) An alert opens a long position held for $H = 20$ days, and the traded position is lagged one day for next-close execution:

$$\text{held}_t = \max_{0 \leq k < H} \text{alert}_{t-k}, \qquad w_t = \text{held}_{t-1} \in \{0,1\}.$$

A.10 Long/flat backtest and performance metrics

With position $w_t \in \{0,1\}$, turnover $u_t = |w_t - w_{t-1}|$, and proportional cost $c = 0.001$ (0.1% per unit turnover), the net strategy return and equity are

$$\rho_t = w_t\, r_t - c\, u_t, \qquad E_t = \prod_{i \leq t} (1 + \rho_i).$$

Writing $Y = (\text{days})/365.25$ for the sample length in years, the reported metrics are

$$\text{multiple} = E_n, \quad \text{CAGR} = E_n^{1/Y} - 1, \quad \text{Sharpe} = \sqrt{365}\,\frac{\bar{\rho}}{\text{std}(\rho)}, \quad \text{MDD} = \min_t \left(\frac{E_t}{\max_{i \leq t} E_i} - 1 \right),$$

together with exposure $= \overline{w}$ (mean position) and the trade count $= \#\{t : w_t > 0,\; w_{t-1} \leq 0\}$ (flat→long transitions). Buy-and-hold uses $w_t \equiv 1$ with a single entry cost.

A.11 Finite-sample Monte Carlo calibration

Asymptotic ADF critical values are unreliable in finite samples, so the ADF-family thresholds are calibrated by simulation. For the asset's own length n and window τ, draw 2000 Gaussian random walks $y^{(b)} = \text{cumsum}(\varepsilon)$ (the unit-root null, seed 123), run the identical `adf_family` routine on each, and record the sample supremum of each series. The empirical 90/95/99th percentiles of those 2000 maxima are the critical values, cached on disk keyed by $(n, \tau, \text{sims}, \text{seed})$ and reused across runs. The count of dates whose statistic exceeds the simulated 95% value is reported as a retrospective significance check (not a per-date p-value).

A.12 Validation: block bootstrap and walk-forward

Two out-of-sample checks guard against selection bias. *Block bootstrap.* Resample the daily return series in contiguous blocks of length 20 (circular indices), 500 times, to obtain confidence intervals on the best in-sample strategy's final multiple and Sharpe, and a White reality-check p-value from the centered excess-over-buy&hold returns,

$$p = \Pr_{\text{boot}}\big[\,\overline{\rho^{\text{boot}} - \rho^{BH}} \;\geq\; \overline{\rho - \rho^{BH}}\,\big].$$

Walk-forward. Split the history into expanding-train / fixed 180-day test blocks (minimum 730 training days); in each block select the signal with the best *training* final multiple, apply it out-of-sample, and concatenate the test-period returns to report an aggregate multiple, Sharpe, and drawdown together with the sequence of selected signals.

B Reference Python Implementation

The listings below are transcribed verbatim from `scripts/generate_daily_regime_chapter.py` at report-build time (via `inspect.getsource`), so the code shown here is exactly the code that produced the numbers in this report. They implement the definitions of Appendix A: the shared regression core, the eight detectors, the causal alert rule, the long/flat backtest and metrics, the Monte Carlo calibration, and the validation checks. Standard scientific-Python imports (`numpy` as `np`, `pandas` as `pd`, `math`) are assumed.

B.1 Shared regression core

`_ols_beta_tstat_intercept_from_sums`

```
def _ols_beta_tstat_intercept_from_sums(
    m: np.ndarray,
    sx: np.ndarray,
    sy: np.ndarray,
    sxx: np.ndarray,
    sxy: np.ndarray,
    syy: np.ndarray,
) -> np.ndarray:
    m = m.astype(float)
    sxx_c = sxx - sx * sx / m
    sxy_c = sxy - sx * sy / m
    syy_c = syy - sy * sy / m
    beta = np.divide(sxy_c, sxx_c, out=np.full_like(sxy_c, np.nan), where=sxx_c > 1e-14)
    sse = syy_c - beta * sxy_c
    df = m - 2
    sigma2 = np.divide(sse, df, out=np.full_like(sse, np.nan), where=df > 0)
    se = np.sqrt(np.divide(sigma2, sxx_c, out=np.full_like(sigma2, np.nan), where=sxx_c > 1e-14))
    return np.divide(beta, se, out=np.full_like(beta, np.nan), where=se > 0)
```

B.2 Indicator statistics

`adf_family`

```
def adf_family(logp: np.ndarray, tau: int) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    n = len(logp)
    x = logp[:-1]
    dy = np.diff(logp)
    px = np.r_[0.0, np.cumsum(x)]
```

```
    py = np.r_[0.0, np.cumsum(dy)]
    pxx = np.r_[0.0, np.cumsum(x * x)]
    pxy = np.r_[0.0, np.cumsum(x * dy)]
    pyy = np.r_[0.0, np.cumsum(dy * dy)]

    sadf = np.full(n, np.nan)
    qadf = np.full(n, np.nan)
    cadf = np.full(n, np.nan)
    for end in range(tau, n):
        starts = np.arange(0, end - tau + 1)
        m = end - starts
        tstats = _ols_beta_tstat_intercept_from_sums(
            m=m,
            sx=px[end] - px[starts],
            sy=py[end] - py[starts],
            sxx=pxx[end] - pxx[starts],
            sxy=pxy[end] - pxy[starts],
            syy=pyy[end] - pyy[starts],
        )
        tstats = tstats[np.isfinite(tstats)]
        if len(tstats) == 0:
            continue
        q = float(np.quantile(tstats, 0.95))
        sadf[end] = float(np.max(tstats))
        qadf[end] = q
        cadf[end] = float(np.mean(tstats[tstats >= q]))
    return sadf, qadf, cadf
```

csw_excess

```
def csw_excess(logp: np.ndarray) -> np.ndarray:
    n = len(logp)
    out = np.full(n, np.nan)
    diff2 = np.diff(logp) ** 2
    prefix = np.r_[0.0, np.cumsum(diff2)]
    for end in range(2, n):
        sigma = math.sqrt(prefix[end] / end) if prefix[end] > 0 else np.nan
        if not np.isfinite(sigma) or sigma <= 0:
            continue
        starts = np.arange(0, end)
        gaps = end - starts
        stat = (logp[end] - logp[starts]) / (sigma * np.sqrt(gaps))
        crit = np.sqrt(4.6 + np.log(gaps))
        out[end] = float(np.nanmax(np.abs(stat) - crit))
    return out
```

recursive_cusum

```
def recursive_cusum(logp: np.ndarray) -> np.ndarray:
    """BDE-inspired␣recursive␣forecast-error␣CUSUM␣with␣online␣normalization."""

    n = len(logp)
    resid = np.full(n, np.nan)
    times = np.arange(n, dtype=float)
    for end in range(8, n):
        y = logp[:end]
        x = times[:end]
        x_mean = x.mean()
        y_mean = y.mean()
        sxx = np.sum((x - x_mean) ** 2)
        if sxx <= 0:
```

```
            continue
        beta = np.sum((x - x_mean) * (y - y_mean)) / sxx
        alpha = y_mean - beta * x_mean
        fitted = alpha + beta * x
        sse = np.sum((y - fitted) ** 2)
        sigma = math.sqrt(sse / max(1, end - 2))
        if sigma <= 0:
            continue
        x0 = times[end]
        h = 1.0 / end + (x0 - x_mean) ** 2 / sxx
        resid[end] = (logp[end] - (alpha + beta * x0)) / (sigma * math.sqrt(1 + h))

    s = pd.Series(resid)
    mu = s.expanding(min_periods=30).mean().shift(1)
    sd = s.expanding(min_periods=30).std(ddof=0).shift(1).replace(0, np.nan)
    z = (s - mu) / sd
    count = z.notna().cumsum()
    csum = z.cumsum()
    out = csum / np.sqrt(count.replace(0, np.nan))
    return out.to_numpy(dtype=float)
```

smt_trend_score

```
def smt_trend_score(logp: np.ndarray, price_norm: np.ndarray, tau: int, phi: float = 0.5) -> np.
    ndarray:
    """SMT-style␣score␣from␣the␣AFML␣17.4.3␣trend␣battery.

␣␣␣␣Covers␣all␣four␣book␣specifications:␣SM-Poly1␣(quadratic␣trend␣on␣raw
␣␣␣␣normalized␣price),␣SM-Poly2␣(quadratic␣trend␣on␣log␣price),␣SM-Exp
␣␣␣␣(linear␣trend␣on␣log␣price),␣and␣SM-Power␣(log␣price␣on␣log␣time).␣Each
␣␣␣␣absolute␣trend␣t-statistic␣is␣penalized␣by␣window␣length␣to␣the␣power
␣␣␣␣``phi``␣before␣taking␣the␣supremum␣over␣backward␣windows.
␣␣␣␣"""

    n = len(logp)
    t = np.arange(1, n + 1, dtype=float)
    logt = np.log(t)
    ones = _prefix(np.ones(n))
    pt = _prefix(t)
    pt2 = _prefix(t**2)
    pt3 = _prefix(t**3)
    pt4 = _prefix(t**4)
    pu = _prefix(logt)
    pu2 = _prefix(logt**2)

    py_log = _prefix(logp)
    pyy_log = _prefix(logp * logp)
    pty_log = _prefix(t * logp)
    pt2y_log = _prefix((t**2) * logp)
    puy_log = _prefix(logt * logp)
    py_pr = _prefix(price_norm)
    pyy_pr = _prefix(price_norm * price_norm)
    pty_pr = _prefix(t * price_norm)
    pt2y_pr = _prefix((t**2) * price_norm)

    def linear_tstat(
        end: int,
        starts: np.ndarray,
        px: np.ndarray,
        pxx: np.ndarray,
        pxy: np.ndarray,
        py: np.ndarray,
        pyy: np.ndarray,
```

```
    ) -> np.ndarray:
        stop = end + 1
        m = ones[stop] - ones[starts]
        return _ols_beta_tstat_intercept_from_sums(
            m,
            px[stop] - px[starts],
            py[stop] - py[starts],
            pxx[stop] - pxx[starts],
            pxy[stop] - pxy[starts],
            pyy[stop] - pyy[starts],
        )

    def quadratic_tstat(
        end: int,
        starts: np.ndarray,
        py: np.ndarray,
        pty: np.ndarray,
        pt2y: np.ndarray,
        pyy: np.ndarray,
    ) -> np.ndarray:
        stop = end + 1
        m = ones[stop] - ones[starts]
        mats = np.empty((len(starts), 3, 3), dtype=float)
        mats[:, 0, 0] = m
        mats[:, 0, 1] = mats[:, 1, 0] = pt[stop] - pt[starts]
        mats[:, 0, 2] = mats[:, 1, 1] = mats[:, 2, 0] = pt2[stop] - pt2[starts]
        mats[:, 1, 2] = mats[:, 2, 1] = pt3[stop] - pt3[starts]
        mats[:, 2, 2] = pt4[stop] - pt4[starts]
        vecs = np.column_stack(
            [
                py[stop] - py[starts],
                pty[stop] - pty[starts],
                pt2y[stop] - pt2y[starts],
            ]
        )
        out = np.full(len(starts), np.nan)
        good = m > 3
        if not np.any(good):
            return out
        try:
            betas = np.linalg.solve(mats[good], vecs[good][..., None]).squeeze(-1)
            fitted_cross = np.sum(betas * vecs[good], axis=1)
            syy = pyy[stop] - pyy[starts[good]]
            sse = syy - fitted_cross
            sigma2 = sse / (m[good] - 3)
            inv = np.linalg.inv(mats[good])
            var = sigma2 * inv[:, 2, 2]
            out[good] = np.abs(betas[:, 2]) / np.sqrt(var)
        except np.linalg.LinAlgError:
            return out
        return out

    score = np.full(n, np.nan)
    for end in range(tau, n):
        starts = np.arange(0, end - tau + 1)
        m = end - starts + 1
        penalty = m.astype(float) ** phi
        exp_stat = np.abs(linear_tstat(end, starts, pt, pt2, pty_log, py_log, pyy_log)) / penalty
        power_stat = np.abs(linear_tstat(end, starts, pu, pu2, puy_log, py_log, pyy_log)) / penalty
        poly_log = quadratic_tstat(end, starts, py_log, pty_log, pt2y_log, pyy_log) / penalty
        poly_price = quadratic_tstat(end, starts, py_pr, pty_pr, pt2y_pr, pyy_pr) / penalty
        vals = np.concatenate([exp_stat, power_stat, poly_log, poly_price])
        vals = vals[np.isfinite(vals)]
        if len(vals):
            score[end] = float(np.max(vals))
```

```
    return score
```

sdfc

```
def sdfc(logp: np.ndarray, tau: int) -> tuple[float, int | None]:
    n = len(logp)
    if n < 2 * tau + 5:
        return np.nan, None
    dy = np.diff(logp)
    lag = logp[:-1]
    best_t = -np.inf
    best_break = None
    obs_index = np.arange(1, n)
    for brk in range(tau, n - tau):
        z = lag * (obs_index >= brk)
        denom = float(np.dot(z, z))
        if denom <= 0:
            continue
        beta = float(np.dot(z, dy) / denom)
        err = dy - beta * z
        sigma2 = float(np.dot(err, err) / max(1, len(dy) - 1))
        se = math.sqrt(sigma2 / denom) if sigma2 > 0 else np.nan
        if se > 0:
            tstat = beta / se
            if tstat > best_t:
                best_t = float(tstat)
                best_break = brk
    if best_break is None:
        return np.nan, None
    return best_t, best_break
```

B.3 Hidden Markov regime filter

_gaussian_diag_loglik

```
def _gaussian_diag_loglik(x: np.ndarray, means: np.ndarray, variances: np.ndarray) -> np.ndarray:
    """Log␣density␣of␣each␣row␣of␣``x``␣under␣each␣diagonal-Gaussian␣state."""

    diff = x[:, None, :] - means[None, :, :]
    quad = np.sum(diff * diff / variances[None, :, :], axis=2)
    norm = np.sum(np.log(2.0 * np.pi * variances), axis=1)
    return -0.5 * (quad + norm[None, :])
```

_hmm_forward_filter

```
def _hmm_forward_filter(
    x: np.ndarray,
    start: np.ndarray,
    trans: np.ndarray,
    means: np.ndarray,
    variances: np.ndarray,
) -> tuple[np.ndarray, float]:
    """Scaled␣forward␣pass.␣Row␣t␣of␣the␣output␣uses␣observations␣0..t␣only.

␣␣␣␣Returns␣the␣filtered␣state␣probabilities␣and␣the␣total␣log-likelihood.
␣␣␣␣"""

    log_b = _gaussian_diag_loglik(x, means, variances)
```

```
    row_max = log_b.max(axis=1)
    b = np.exp(log_b - row_max[:, None])
    n, k = b.shape
    alpha = np.empty((n, k))
    scale = np.empty(n)
    a = start * b[0]
    scale[0] = max(a.sum(), 1e-300)
    alpha[0] = a / scale[0]
    for t in range(1, n):
        a = (alpha[t - 1] @ trans) * b[t]
        scale[t] = max(a.sum(), 1e-300)
        alpha[t] = a / scale[t]
    loglik = float(np.sum(np.log(scale)) + np.sum(row_max))
    return alpha, loglik
```

fit_gaussian_hmm

```
def fit_gaussian_hmm(
    x: np.ndarray,
    n_states: int,
    *,
    n_iter: int = DEFAULT_HMM_EM_ITER,
    tol: float = 1e-6,
) -> dict | None:
    """Deterministic␣EM␣fit␣of␣a␣diagonal-covariance␣Gaussian␣HMM.

␣␣␣␣Initialization␣is␣deterministic␣(states␣seeded␣from␣quantile␣slices␣of␣the
␣␣␣␣first␣feature),␣so␣repeated␣fits␣on␣identical␣data␣give␣identical␣models.
␣␣␣␣Returns␣``None``␣when␣the␣sample␣is␣too␣short␣or␣the␣fit␣degenerates.
␣␣␣␣"""

    x = np.asarray(x, dtype=float)
    n, d = x.shape
    if n < max(8 * n_states, 2 * d + 2) or not np.isfinite(x).all():
        return None
    var_floor = np.maximum(np.var(x, axis=0), 1e-12) * 1e-4 + 1e-12

    order = np.argsort(x[:, 0], kind="mergesort")
    means = np.empty((n_states, d))
    variances = np.empty((n_states, d))
    for state, chunk in enumerate(np.array_split(order, n_states)):
        means[state] = x[chunk].mean(axis=0)
        variances[state] = np.maximum(x[chunk].var(axis=0), var_floor)
    start = np.full(n_states, 1.0 / n_states)
    if n_states > 1:
        trans = np.full((n_states, n_states), 0.1 / (n_states - 1))
        np.fill_diagonal(trans, 0.9)
    else:
        trans = np.ones((1, 1))

    prev_loglik = -np.inf
    for _ in range(n_iter):
        log_b = _gaussian_diag_loglik(x, means, variances)
        row_max = log_b.max(axis=1)
        b = np.exp(log_b - row_max[:, None])

        alpha = np.empty((n, n_states))
        scale = np.empty(n)
        a = start * b[0]
        scale[0] = max(a.sum(), 1e-300)
        alpha[0] = a / scale[0]
        for t in range(1, n):
            a = (alpha[t - 1] @ trans) * b[t]
```

```
            scale[t] = max(a.sum(), 1e-300)
            alpha[t] = a / scale[t]

        beta = np.empty((n, n_states))
        beta[-1] = 1.0
        for t in range(n - 2, -1, -1):
            beta[t] = (trans @ (b[t + 1] * beta[t + 1])) / scale[t + 1]

        gamma = alpha * beta
        gamma /= np.maximum(gamma.sum(axis=1, keepdims=True), 1e-300)
        xi_sum = trans * (alpha[:-1].T @ ((b[1:] * beta[1:]) / scale[1:, None]))
        loglik = float(np.sum(np.log(scale)) + np.sum(row_max))

        start = np.maximum(gamma[0], 1e-12)
        start /= start.sum()
        trans = np.maximum(xi_sum, 1e-12)
        trans /= trans.sum(axis=1, keepdims=True)
        weights = np.maximum(gamma.sum(axis=0), 1e-8)
        means = (gamma.T @ x) / weights[:, None]
        variances = np.maximum((gamma.T @ (x * x)) / weights[:, None] - means**2, var_floor)

        if not np.isfinite(loglik):
            return None
        if abs(loglik - prev_loglik) < tol * max(1.0, abs(prev_loglik)):
            prev_loglik = loglik
            break
        prev_loglik = loglik

    # Final E-step responsibilities under the converged parameters.
    log_b = _gaussian_diag_loglik(x, means, variances)
    row_max = log_b.max(axis=1)
    b = np.exp(log_b - row_max[:, None])
    alpha = np.empty((n, n_states))
    scale = np.empty(n)
    a = start * b[0]
    scale[0] = max(a.sum(), 1e-300)
    alpha[0] = a / scale[0]
    for t in range(1, n):
        a = (alpha[t - 1] @ trans) * b[t]
        scale[t] = max(a.sum(), 1e-300)
        alpha[t] = a / scale[t]
    beta = np.empty((n, n_states))
    beta[-1] = 1.0
    for t in range(n - 2, -1, -1):
        beta[t] = (trans @ (b[t + 1] * beta[t + 1])) / scale[t + 1]
    gamma = alpha * beta
    gamma /= np.maximum(gamma.sum(axis=1, keepdims=True), 1e-300)

    return {
        "start": start,
        "trans": trans,
        "means": means,
        "variances": variances,
        "gamma": gamma,
        "loglik": prev_loglik,
    }
```

hmm_regime_signal

```
def hmm_regime_signal(
    logp: np.ndarray,
    *,
    n_states: int = DEFAULT_HMM_STATES,
```

```
    refit_days: int = DEFAULT_HMM_REFIT_DAYS,
    min_history: int = DEFAULT_HMM_MIN_HISTORY,
    momentum_days: int = DEFAULT_MOMENTUM_DAYS,
    n_iter: int = DEFAULT_HMM_EM_ITER,
) -> tuple[np.ndarray, np.ndarray]:
    """Walk-forward Gaussian HMM regime filter with no forward-looking bias.

    Design guarantees against lookahead:

    - Features at close ``t`` (daily log return and ``momentum_days`` log
      momentum) use prices through ``t`` only.
    - Model parameters used for close ``t`` are fitted on features strictly
      before the most recent refit date, which is itself ``<= t``.
    - The regime at close ``t`` is the argmax of *filtered* probabilities
      from a forward pass over features ``0..t`` (never smoothed with future
      observations).
    - The regime-to-signal map (which state is bullish) is computed from the
      training window only, using EM responsibilities against same-window
      returns.
    - Refit dates are fixed absolute indices, so the value at ``t`` does not
      change when future data is appended.

    Returns a boolean bull-signal array and an integer state array (``-1``
    where no model is available), both aligned to ``logp``.
    """

    n = len(logp)
    signal = np.zeros(n, dtype=bool)
    states = np.full(n, -1, dtype=int)
    if n <= momentum_days + 1:
        return signal, states

    t_idx = np.arange(momentum_days, n)
    features = np.column_stack(
        [
            logp[t_idx] - logp[t_idx - 1],
            logp[t_idx] - logp[t_idx - momentum_days],
        ]
    )
    n_feat = len(features)
    if n_feat <= min_history:
        return signal, states

    model: dict | None = None
    bull_state: int | None = None
    fit_at = min_history
    while fit_at < n_feat:
        block_end = min(fit_at + refit_days, n_feat)
        fitted = fit_gaussian_hmm(features[:fit_at], n_states, n_iter=n_iter)
        if fitted is not None:
            model = fitted
            gamma = fitted["gamma"]
            weights = np.maximum(gamma.sum(axis=0), 1e-8)
            state_mean_return = (gamma * features[:fit_at, [0]]).sum(axis=0) / weights
            best = int(np.argmax(state_mean_return))
            bull_state = best if state_mean_return[best] > 0 else None
        if model is not None:
            filtered, _ = _hmm_forward_filter(
                features[:block_end],
                model["start"],
                model["trans"],
                model["means"],
                model["variances"],
            )
            block_states = np.argmax(filtered[fit_at:block_end], axis=1)
```

```
            states[t_idx[fit_at:block_end]] = block_states
            if bull_state is not None:
                signal[t_idx[fit_at:block_end]] = block_states == bull_state
        fit_at = block_end
    return signal, states
```

B.4 Alert rule and signal pipeline

online_high_signal

```
def online_high_signal(
    values: np.ndarray,
    *,
    use_abs: bool = False,
    q: float = DEFAULT_SIGNAL_Q,
    min_history: int = DEFAULT_SIGNAL_MIN_HISTORY,
) -> pd.Series:
    series = pd.Series(values).replace([np.inf, -np.inf], np.nan)
    target = series.abs() if use_abs else series
    threshold = target.expanding(min_periods=min_history).quantile(q).shift(1)
    return (target > threshold).fillna(False)
```

compute_strategy_suite

```
def compute_strategy_suite(
    asset: str,
    dates: pd.Series,
    close_array: np.ndarray,
    logp: np.ndarray,
    bde: np.ndarray,
    csw: np.ndarray,
    sadf: np.ndarray,
    qadf: np.ndarray,
    cadf: np.ndarray,
    smt: np.ndarray,
    hmm_bull: np.ndarray,
    config: RunConfig,
    *,
    write_csv: bool = True,
    include_validation: bool = True,
) -> tuple[dict, str]:
    close = pd.Series(close_array, dtype=float)
    momentum = pd.Series(logp).diff(config.momentum_days).gt(0).fillna(False)
    raw_events = {
        "BDE": online_high_signal(
            bde, use_abs=True, q=config.signal_q, min_history=config.signal_min_history
        )
        & momentum,
        "CSW": pd.Series(csw).gt(0).fillna(False) & momentum,
        "SADF": online_high_signal(sadf, q=config.signal_q, min_history=config.signal_min_history) &
            momentum,
        "QADF": online_high_signal(qadf, q=config.signal_q, min_history=config.signal_min_history) &
            momentum,
        "CADF": online_high_signal(cadf, q=config.signal_q, min_history=config.signal_min_history) &
            momentum,
        "SMT": online_high_signal(smt, q=config.signal_q, min_history=config.signal_min_history) &
            momentum,
    }
    consensus_event = (
        raw_events["BDE"].astype(int)
```

```
    + raw_events["CSW"].astype(int)
    + raw_events["SADF"].astype(int)
    + raw_events["SMT"].astype(int)
) >= 2
raw_events["Consensus"] = consensus_event

strategy_frame = pd.DataFrame(
    {
        "date": dates.dt.strftime("%Y-%m-%d"),
        "close": close,
        "return": close.pct_change().fillna(0.0),
        "momentum_20d_positive": momentum,
    }
)
metrics: dict[str, dict] = {}

buyhold_position = pd.Series(1.0, index=close.index)
buyhold_returns, buyhold_equity, buyhold_metrics = backtest_position(
    dates, close, buyhold_position, transaction_cost=config.transaction_cost, charge_costs=True
)
strategy_frame["pos_BuyHold"] = buyhold_position
strategy_frame["ret_BuyHold"] = buyhold_returns
strategy_frame["equity_BuyHold"] = buyhold_equity
metrics["BuyHold"] = buyhold_metrics

for name, event in raw_events.items():
    held = event.rolling(config.hold_days, min_periods=1).max().astype(bool)
    position = held.shift(1).fillna(False).astype(float)
    strategy_returns, equity, stats = backtest_position(
        dates, close, position, transaction_cost=config.transaction_cost
    )
    strategy_frame[f"event_{name}"] = event.astype(int)
    strategy_frame[f"pos_{name}"] = position
    strategy_frame[f"ret_{name}"] = strategy_returns
    strategy_frame[f"equity_{name}"] = equity
    metrics[name] = stats

# HMM is a persistent regime state, not an alert event: hold the position
# exactly while the filtered bull state is active, with the same one-day
# execution shift as the alert strategies. No hold window or momentum gate.
hmm_event = pd.Series(np.asarray(hmm_bull, dtype=bool), index=close.index)
hmm_position = hmm_event.shift(1).fillna(False).astype(float)
hmm_returns, hmm_equity, hmm_stats = backtest_position(
    dates, close, hmm_position, transaction_cost=config.transaction_cost
)
strategy_frame["event_HMM"] = hmm_event.astype(int)
strategy_frame["pos_HMM"] = hmm_position
strategy_frame["ret_HMM"] = hmm_returns
strategy_frame["equity_HMM"] = hmm_equity
metrics["HMM"] = hmm_stats

best_signal = max(STRUCTURE_STRATEGIES, key=lambda name: metrics[name]["final_multiple"])
best_including_buyhold = max(STRATEGY_NAMES, key=lambda name: metrics[name]["final_multiple"])
out_csv = DATA_DIR / f"{asset}_strategy_daily.csv"
if write_csv:
    strategy_frame.to_csv(out_csv, index=False)

summary = {
    "strategy_file": str(out_csv.relative_to(ROOT)) if write_csv else "",
    "parameters": {
        "transaction_cost": config.transaction_cost,
        "signal_quantile": config.signal_q,
        "signal_min_history": config.signal_min_history,
        "momentum_days": config.momentum_days,
        "hold_days": config.hold_days,
```

```
            "hmm_states": config.hmm_states,
            "hmm_refit_days": config.hmm_refit_days,
            "hmm_min_history": config.hmm_min_history,
            "buyhold_entry_cost_charged": True,
        },
        "metrics": metrics,
        "best_signal_strategy": best_signal,
        "best_signal_final_multiple": metrics[best_signal]["final_multiple"],
        "best_including_buyhold": best_including_buyhold,
        "best_including_buyhold_final_multiple": metrics[best_including_buyhold]["final_multiple"],
    }
    if include_validation:
        summary["walk_forward"] = walk_forward_evaluation(strategy_frame, dates, config)
        summary["multiple_testing"] = bootstrap_multiple_testing(strategy_frame, config)
    return summary, str(out_csv.relative_to(ROOT)) if write_csv else ""
```

B.5 Finite-sample Monte Carlo calibration

`_simulate_adf_worker`

```
def _simulate_adf_worker(args: tuple[int, int, int, int]) -> dict[str, list[float]]:
    n, tau, sims, seed = args
    rng = np.random.default_rng(seed)
    out = {"sadf": [], "qadf": [], "cadf": []}
    for _ in range(sims):
        y = np.cumsum(rng.normal(size=n))
        sadf, qadf, cadf = adf_family(y, tau)
        out["sadf"].append(finite_nanmax(sadf))
        out["qadf"].append(finite_nanmax(qadf))
        out["cadf"].append(finite_nanmax(cadf))
    return out
```

`simulate_adf_critical_values`

```
def simulate_adf_critical_values(n: int, tau: int, config: RunConfig) -> dict:
    sims = int(config.calibration_sims)
    seed = int(config.calibration_seed)
    if sims <= 0:
        return {
            "version": CALIBRATION_VERSION,
            "n": n,
            "tau": tau,
            "sims": 0,
            "seed": seed,
            "critical_values": {},
        }

    CALIBRATION_DIR.mkdir(parents=True, exist_ok=True)
    path = calibration_cache_path(n, tau, sims, seed)
    if path.exists():
        return json.loads(path.read_text())

    workers = max(1, min(config.workers, sims))
    counts = [sims // workers] * workers
    for i in range(sims % workers):
        counts[i] += 1
    tasks = [(n, tau, count, seed + 1009 * (i + 1)) for i, count in enumerate(counts) if count > 0]
    if workers == 1:
        parts = [_simulate_adf_worker(task) for task in tasks]
    else:
```

```
        with Pool(processes=workers) as pool:
            parts = pool.map(_simulate_adf_worker, tasks)

    combined = {"sadf": [], "qadf": [], "cadf": []}
    for part in parts:
        for key in combined:
            combined[key].extend(part[key])

    critical_values = {}
    for key, values in combined.items():
        arr = np.asarray(values, dtype=float)
        arr = arr[np.isfinite(arr)]
        critical_values[f"{key}_90"] = float(np.quantile(arr, 0.90)) if len(arr) else np.nan
        critical_values[f"{key}_95"] = float(np.quantile(arr, 0.95)) if len(arr) else np.nan
        critical_values[f"{key}_99"] = float(np.quantile(arr, 0.99)) if len(arr) else np.nan

    payload = {
        "version": CALIBRATION_VERSION,
        "n": n,
        "tau": tau,
        "sims": sims,
        "seed": seed,
        "critical_values": critical_values,
    }
    path.write_text(json.dumps(strict_json_value(payload), indent=2, allow_nan=False))
    return payload
```

B.6 Long/flat backtest and metrics

max_drawdown

```
def max_drawdown(equity: pd.Series) -> float:
    running_max = equity.cummax()
    drawdown = equity / running_max - 1.0
    return float(drawdown.min())
```

metrics_from_returns

```
def metrics_from_returns(
    strategy_returns: pd.Series,
    *,
    days: int | None = None,
    position: pd.Series | None = None,
) -> tuple[pd.Series, dict]:
    strategy_returns = strategy_returns.astype(float).fillna(0.0)
    equity = (1.0 + strategy_returns).cumprod()
    days = int(days if days is not None else max(1, len(strategy_returns) - 1))
    years = max(days / 365.25, 1 / 365.25)
    daily_std = float(strategy_returns.std(ddof=0))
    sharpe = float(np.sqrt(365.0) * strategy_returns.mean() / daily_std) if daily_std > 0 else np.
        nan
    metrics = {
        "final_multiple": float(equity.iloc[-1]) if len(equity) else np.nan,
        "total_return": float(equity.iloc[-1] - 1.0) if len(equity) else np.nan,
        "cagr": float(equity.iloc[-1] ** (1.0 / years) - 1.0)
        if len(equity) and years > 0 and equity.iloc[-1] > 0
        else np.nan,
        "max_drawdown": max_drawdown(equity) if len(equity) else np.nan,
        "sharpe": sharpe,
        "exposure": float(position.mean()) if position is not None else np.nan,
```

```
        "trades": int(((position > 0) & (position.shift(1).fillna(0) <= 0)).sum()) if position is not
            None else 0,
        "turnover": float(position.diff().abs().fillna(position.abs()).sum()) if position is not None
            else np.nan,
    }
    return equity, metrics
```

backtest_position

```
def backtest_position(
    dates: pd.Series,
    close: pd.Series,
    position: pd.Series,
    *,
    transaction_cost: float = DEFAULT_TRANSACTION_COST,
    charge_costs: bool = True,
) -> tuple[pd.Series, pd.Series, dict]:
    returns = close.pct_change().fillna(0.0)
    position = position.astype(float).fillna(0.0)
    turnover = position.diff().abs().fillna(position.abs())
    costs = transaction_cost * turnover if charge_costs else 0.0
    strategy_returns = position * returns - costs
    days = max(1, int((pd.Timestamp(dates.iloc[-1]) - pd.Timestamp(dates.iloc[0])).days))
    equity, metrics = metrics_from_returns(strategy_returns, days=days, position=position)
    return strategy_returns, equity, metrics
```

B.7 Validation: block bootstrap and walk-forward

block_bootstrap_indices

```
def block_bootstrap_indices(n: int, block_size: int, rng: np.random.Generator) -> np.ndarray:
    if n <= 0:
        return np.array([], dtype=int)
    starts = rng.integers(0, n, size=int(math.ceil(n / block_size)))
    pieces = [(start + np.arange(block_size)) % n for start in starts]
    return np.concatenate(pieces)[:n]
```

bootstrap_multiple_testing

```
def bootstrap_multiple_testing(strategy_frame: pd.DataFrame, config: RunConfig) -> dict:
    sims = config.bootstrap_sims
    if sims <= 0:
        return {"sims": 0, "note": "bootstrap disabled"}
    n = len(strategy_frame)
    rng = np.random.default_rng(config.bootstrap_seed + n)
    buyhold = strategy_frame["ret_BuyHold"].to_numpy(dtype=float)
    structure_returns = {
        name: strategy_frame[f"ret_{name}"].to_numpy(dtype=float) for name in STRUCTURE_STRATEGIES
    }
    final_multiples = {
        name: float(np.prod(1.0 + values)) for name, values in structure_returns.items()
    }
    best_signal = max(final_multiples, key=final_multiples.get)
    best_returns = structure_returns[best_signal]
    excess = np.column_stack([structure_returns[name] - buyhold for name in STRUCTURE_STRATEGIES])
    observed_best_mean_excess = float(np.nanmax(excess.mean(axis=0)))
    centered_excess = excess - excess.mean(axis=0)
```

```
    boot_final = []
    boot_sharpe = []
    boot_reality = []
    for _ in range(sims):
        idx = block_bootstrap_indices(n, max(2, config.hold_days), rng)
        sampled = best_returns[idx]
        boot_final.append(float(np.prod(1.0 + sampled)))
        std = float(np.std(sampled, ddof=0))
        boot_sharpe.append(float(np.sqrt(365.0) * np.mean(sampled) / std) if std > 0 else np.nan)
        boot_reality.append(float(np.nanmax(centered_excess[idx].mean(axis=0))))

    boot_final_arr = np.asarray(boot_final, dtype=float)
    boot_sharpe_arr = np.asarray(boot_sharpe, dtype=float)
    boot_reality_arr = np.asarray(boot_reality, dtype=float)
    return {
        "sims": int(sims),
        "block_size": int(max(2, config.hold_days)),
        "best_signal": best_signal,
        "best_signal_final_multiple": final_multiples[best_signal],
        "best_signal_final_ci_05": float(np.nanquantile(boot_final_arr, 0.05)),
        "best_signal_final_ci_95": float(np.nanquantile(boot_final_arr, 0.95)),
        "best_signal_sharpe_ci_05": float(np.nanquantile(boot_sharpe_arr, 0.05)),
        "best_signal_sharpe_ci_95": float(np.nanquantile(boot_sharpe_arr, 0.95)),
        "observed_best_daily_excess_mean": observed_best_mean_excess,
        "reality_check_p_value": float(np.mean(boot_reality_arr >= observed_best_mean_excess)),
    }
```

walk_forward_evaluation

```
def walk_forward_evaluation(strategy_frame: pd.DataFrame, dates: pd.Series, config: RunConfig) ->
    dict:
    n = len(strategy_frame)
    start = config.walk_forward_train_min
    blocks = []
    selected_returns = []
    selected_dates = []

    while start + config.walk_forward_test_size <= n:
        train = slice(0, start)
        test = slice(start, start + config.walk_forward_test_size)
        train_multiples = {
            name: float(np.prod(1.0 + strategy_frame[f"ret_{name}"].iloc[train].to_numpy(dtype=float)
                ))
            for name in STRUCTURE_STRATEGIES
        }
        selected = max(train_multiples, key=train_multiples.get)
        test_returns = strategy_frame[f"ret_{selected}"].iloc[test].reset_index(drop=True)
        test_days = max(
            1,
            int(
                (
                    pd.Timestamp(dates.iloc[start + config.walk_forward_test_size - 1])
                    - pd.Timestamp(dates.iloc[start])
                ).days
            ),
        )
        _, test_metrics = metrics_from_returns(test_returns, days=test_days)
        blocks.append(
            {
                "train_end": pd.Timestamp(dates.iloc[start - 1]).date().isoformat(),
                "test_start": pd.Timestamp(dates.iloc[start]).date().isoformat(),
                "test_end": pd.Timestamp(dates.iloc[start + config.walk_forward_test_size - 1]).date()
                    .isoformat(),
```

```
            "selected_signal": selected,
            "test_final_multiple": test_metrics["final_multiple"],
            "test_sharpe": test_metrics["sharpe"],
            "test_max_drawdown": test_metrics["max_drawdown"],
        }
    )
    selected_returns.append(test_returns)
    selected_dates.extend(dates.iloc[test].tolist())
    start += config.walk_forward_test_size

if not selected_returns:
    return {
        "blocks": [],
        "aggregate": {
            "blocks": 0,
            "test_days": 0,
            "final_multiple": np.nan,
            "sharpe": np.nan,
            "max_drawdown": np.nan,
            "selected_signals": "",
        },
    }

combined = pd.concat(selected_returns, ignore_index=True)
test_days = max(1, int((pd.Timestamp(selected_dates[-1]) - pd.Timestamp(selected_dates[0])).days)
    )
_, aggregate = metrics_from_returns(combined, days=test_days)
aggregate.update(
    {
        "blocks": len(blocks),
        "test_days": int(len(combined)),
        "selected_signals": ",␣".join(block["selected_signal"] for block in blocks),
    }
)
return {"blocks": blocks, "aggregate": aggregate}
```